



Module Code:

CSIS 4495 – 071

Module Title:

Applied Research Project

Year & Semester:

2025 Fall

Project Midterm Report

Alternative Industries: Weather Driver

Student ID	Student Name
300392983	Pukar Ojha
300391004	Chetan Kaur (Team Lead)
300392663	Pravesh Poudel

Instructor: Bambang, Sarif

Due Date: Oct 25th, 2025

Table of Contents

Introduction.....	1
Problem Statement	1
Literature Context and Gaps	2
Project as a Solution.....	2
Assumptions & Hypotheses.....	2
Benefits	2
Proposed Research Project.....	3
Design and Objectives	3
Aims.....	3
Objectives	3
Methodology	3
Considered Methodologies	3
Selected Methodology: Agile with Scrum	4
Why Agile with Scrum.....	4
Phases of Agile.....	4
Initiate	4
Plan and Estimate.....	5
Implement	5
Review and Retrospect	5
Release	6

Expected Outcomes and Deliverables	6
Expected Outcomes	6
Key Deliverables.....	6
Resources Required	7
Human Resources	7
Technologies And Tools.....	7
Other Resources	7
Project planning and Timeline	8
Roles & Responsibilities.....	8
Work Breakdown Structure (WBS)	8
Milestones	9
Gantt Charts	9
Implemented Feature 1: User Authentication and Onboarding System.....	11
Overview.....	11
Implementation Details.....	11
Technical Implementation.....	26
Summary	28
Implemented Feature 2: Core Navigation and Map Interface	29
Overview.....	29
Implementation Details	29
Main Navigation Structure (App.js):	29

Tab Screens:	30
Map Interface Features	38
Technical Architecture	40
Collaboration Highlights.....	41
Summary	41
Implemented Feature 3: Settings Management System.....	42
Overview.....	42
Implementation Details.....	42
Technical Features	43
Gas Station Settings	43
Code Quality and Organization	48
Testing and Validation.....	48
Implemented Feature 4: Speedometer and Gas Station Preferences	49
Speedometer Screen.....	49
(SpeedometerScreen.js):	49
Features Implemented:.....	49
Technical Details:.....	49
Splash Screen Refinement	52
Lessons Learned and Future Work.....	53
Lessons Learned.....	53
Future Work	54

Concluding Remarks.....	56
Appendix.....	57
Appendix A: Installation Guide	57
Appendix B: User Guide.....	59
Appendix C: Dataset and APIs Used	60
Appendix D: Hardware, Software, and System Architecture	61
Appendix E: Backend Infrastructure and AWS Integration Research	62
Appendix F: Challenges and solutions	65
Appendix G: Next Steps and Future Work	69
Appendix H: Code Explanation	73
References.....	111

Introduction

Driving is not just about getting from one place to another it is also about staying safe and being prepared for unexpected conditions on the road. Severe weather, limited fuel stops, and sudden hazards can all create risks for drivers if they are not warned in time. While navigation apps like Google Maps provide directions and traffic updates, they rarely integrate real time weather alerts or extra driver-focused tools in a single platform.

The Weather Driver mobile app is designed to close this gap. It combines navigation with live hazardous weather notifications, giving users an added layer of safety while traveling. Beyond navigation and weather, the app also includes practical features such as gas station locators, a built-in speedometer, toll pass tracking, customizable notifications, and user-friendly settings. These additions make the app not just a route planner, but a complete driver's companion.

Studies on transportation and road safety apps show that it's very important for drivers to get live updates about road hazards. The problem is that many existing solutions only work with expensive car systems or are not easy for everyday drivers to use. Our project is based on the idea that a mobile app can make these features more accessible, helping drivers stay more aware, confident, and prepared while traveling. The benefit of this work is twofold: it can make driving safer by reducing risks from sudden weather changes, and it can also make the overall driving experience easier by bringing several useful tools together in one app.

Problem Statement

Road accidents and travel disruptions due to severe weather conditions are a growing concern. According to the U.S. Department of Transportation, nearly 21% of vehicle crashes annually are weather-related, leading to thousands of deaths and injuries each year. Current navigation apps like Google Maps offer route planning but lack integrated real-time hazardous weather alerts. (U.S. Department of Transportation, Federal Highway Administration, 2025)

Literature Context and Gaps

Research on transportation and road safety apps highlights the importance of live hazard updates for drivers. However, existing solutions are either tied to expensive in-car systems or fail to provide a seamless mobile experience. This project addresses that gap by making safety-focused features more accessible via a mobile app.

Project as a Solution

The weather driver mobile app is designed to make driving safer by combining Google Maps navigation with real-time weather hazard alerts. The app not only provides directions through Google Maps but also delivers timely warnings about severe weather conditions and potential hazards. By receiving these alerts, users can make safer route choices during their trips. In addition, the app will be hosted on AWS to ensure it remains scalable, reliable, and accessible anytime.

Assumptions & Hypotheses

Several assumptions and hypotheses have been considered while working on the Weather Driver project. We assume that mobile app users will benefit from receiving real-time weather hazard alerts integrated with navigation. By combining Google Maps with weather APIs such as the National Oceanic and Atmospheric Administration (NOAA), the app is expected to reduce travel risks and support safer decision-making. Furthermore, hosting the cross-platform mobile application on AWS will ensure scalable, reliable, and consistent access to these services.

Benefits

This solution will improve driver safety, increase weather awareness, and provide a foundation for future growth through ad revenue and feature expansion.

Proposed Research Project

Design and Objectives

Our project is designed as applied research, which means we are building something practical that will be tested in the real world. The main goals are to create a mobile app that combines navigation with live weather hazard alerts, expands it with useful features like gas stations, toll passes, a speedometer, settings, and notifications, and ensures it runs smoothly and reliably.

Aims

To develop and deploy a mobile app that enhances road safety by integrating navigation with real-time weather warnings.

Objectives

A project should have clear objectives going forward, similarly we discussed objectives with the client and described them as below:

- Implement frontend in React Native using existing Figma designs.
- Integrate Google Maps API for directions and hazard overlays.
- Add weather API (NOAA) integration for real-time alerts.
- Connect to existing backend and database.
- Deploy on AWS for scalability and availability.
- Deliver a working app with technical and user documentation.

Methodology

Considered Methodologies

Before initiating the project, a software methodology is important to consider successful completion of the project. In this project, we have considered several methodologies, and they are waterfall methodologies which have clear structure but are inflexible for mid-project changes and second is agile methodologies which is Iterative, ideal for student projects.

Selected Methodology: Agile with Scrum

We chose the Agile methodology because it focuses on flexibility, teamwork, and delivering results step by step. To implement Agile, we will follow the Scrum framework, which organizes our work into sprints. In each sprint, we will plan tasks, develop features, test them, and then review what we achieved. Scrum also emphasizes roles and ceremonies:

Scrum Roles: Our team will act as the development team, with one person taking on the role of Scrum Master to keep meetings and progress on track, another as Product Owner responsible for prioritizing features from the backlog and the development team.

Scrum Ceremonies: We will have short weekly standups to share progress, sprint planning meetings to assign tasks, sprint reviews to demo completed work, and retrospectives to discuss improvements.

Why Agile with Scrum

Scrum is well-suited for our project because it allows us to focus on one feature at a time (like maps first, then weather, then gas stations, and so on), while still delivering working parts of the app regularly. It also helps us stay flexible if we face technical issues with APIs or integrations, since we can adjust priorities for the next sprint. With three team members, Scrum's structure makes collaboration clearer, since each of us will have assigned tasks but still work closely together. (scrumstudy.com, 2025)

Phases of Agile

Our Scrum-based Agile process will still follow the main SDLC phases, but in an iterative loop:

Initiate

In this phase, we will set the foundation of the project. Our project vision is to build a driver-focused mobile app that combines navigation with real-time weather hazard alerts and other helpful tools. Within our team, one member will take the role of Product Owner to manage the backlog and priorities; another will be the Scrum Master to facilitate meetings, and all three of us will work as the development team. We will start by defining the main features, such as

navigation, weather alerts, gas stations, toll passes, notifications, and a speedometer, then organize them into a prioritized backlog and plan the release to be completed within our 10-week timeline.

Plan and Estimate

In this phase, we will define the main Epics for our project, such as navigation, weather alerts, gas stations, toll passes, notifications, and a speedometer. These Epics represent the big features of the app and will guide how we plan and organize our sprints.

Implement

This is where development happens. Each sprint (2 weeks) will focus on completing a set of features, along with unit testing to ensure they work correctly:

Sprint 0: Team Formation, Defining the project Scope, goals, and objectives, defining team roles, setting up the tools like JIRA, and working on the Project Proposal.

Sprint 1: Set up the environment, repository, and AWS, implement navigation with Google Maps, analyze the existing system architecture, and adjust the screen color (as per the Figma screens).

Sprint 2: Integrating the weather of API and hazard alerts, performing unit tests for API responses and alert displays, and creating the user profile.

Sprint 3: Add features such as gas station locator, toll passes tracking, with unit tests for each new feature.

Sprint 4: Developing the notifications and settings of screens, integrating them with the backend, and performing unit tests to ensure notification triggers and user preferences work correctly.

Sprint 5: Carry out final user acceptance testing, fix bugs, complete final deployment, and prepare documentation and product handover to the client.

We will hold weekly standups to share progress and refine the backlog. (scrumstudy.com, 2025)

Review and Retrospect

The project is divided into five sprints, and each sprint will include both a review and a retrospective. At the end of every sprint, the team will conduct a Sprint Review to demonstrate the completed features and gather feedback from teammates, peers, and instructor. This will

be followed by a Sprint Retrospective, where the team will reflect on what went well, identify any challenges faced, and discuss strategies for improvement in the upcoming sprint.

Release

While the sprints focus on development, a project is not complete until it is properly delivered and released to the client. Before the handover, tasks such as hosting the project on AWS, building test cases, and preparing documentation will be carried out to ensure a successful release. As part of the release phase, the team will deploy the working app on AWS and generate a test build for users. The final delivery will include key features such as navigation, weather alerts, settings, notifications, and gas stations. In addition, both user documentation and technical documentation will be prepared to support future use and maintenance. Finally, the team will reflect on the lessons learned throughout the project, using these insights to guide future improvements or expansions of the app.

Expected Outcomes and Deliverables

Expected Outcomes

Before starting a project, a team should have clear understanding of its aims and objectives also the expected outcomes, which are directed towards the successful project and our expected outcomes are as: a mobile app that alerts drivers about weather hazards, seamless integration of mapping and real-time weather data and a scalable and deployable solution hosted on AWS.

Key Deliverables

In discussions with the client, the key deliverables for the project were clearly outlined. These include the implementation of the front end using React Native, incorporating core features such as navigation, location search, and hazard or weather warnings. The app will integrate Google Maps with weather APIs to provide real-time information for safer travel. Additionally, deployment scripts and setup will be prepared for AWS hosting to ensure scalability and reliability. The outcome will be a fully functional mobile application, delivered as an APK or test build. Alongside the application, comprehensive technical documentation, including setup guides and API references, will be provided, as well as user documentation to assist with app usage.

Resources Required

Human Resources

A successful project requires adequate manpower to ensure timely completion. For this project, the team is composed of various roles, including React Native Developers, a Project Mentor, a UI/UX Designer, and a Backend Developer, each contributing their expertise to achieve the project's objectives.

Technologies And Tools

The key part of building a software/ app is understanding the technologies and tools to be used for development. Here, we discussed the tools we'll be using for the project.

- React Native
- Expo, GitHub, Postman
- Google Maps API, Weather APIs
- Node.js/Express (backend)
- MongoDB/MySQL (DB)
- AWS (EC2, S3, SQS)

Other Resources

The development process involves more than just writing code, it also requires proper documentation, design resources, a suitable development environment, and mentorship. The team will rely on existing UI/UX documentation to guide design decisions, while development environments will include both local machines and AWS accounts for building and testing the application. In addition, weekly consultations with the project mentor will provide valuable guidance and support throughout the development cycle.

Project planning and Timeline

Roles & Responsibilities

The team consists of three members, and their roles and responsibilities are listed below.

- **Chetan Kaur (Team Lead):** Coordination, frontend, documentation, QA, Scrum Master.
- **Pravesh Poudel:** Frontend, Ui Design, Backend and DB assistance, Product Owner.
- **Pukar Ojha:** API integration, testing, backend integration, and Database implementation.

Work Breakdown Structure (WBS)

A Work Breakdown Structure is a method of breaking down the overall project into smaller, more manageable parts. It organizes the project into tasks and sub-tasks, making it easier to assign responsibilities, estimate timelines, and track progress.

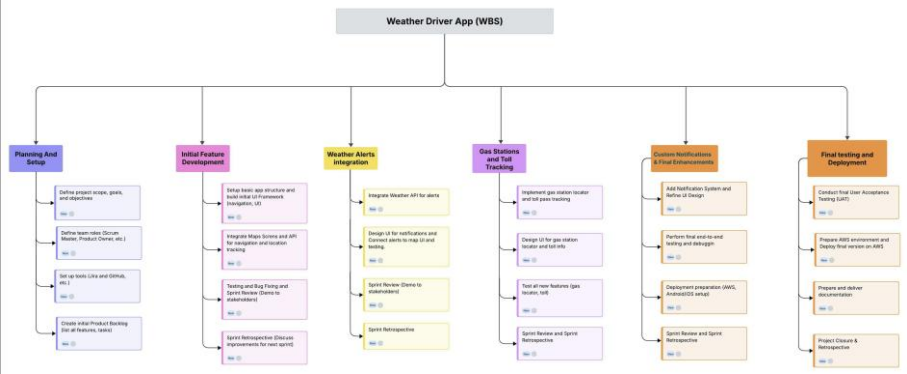


Figure 1: Work Breakdown Structure

Milestones

Milestone can be simply known as important events. A chart that represents important events planned in a timeline is known as a milestone chart. The milestone chart of the wager app is constructed to mark important events with proper timelines. Many tasks set the proper timeline for timely completion of the project and ease the flow of the project and development methodology.

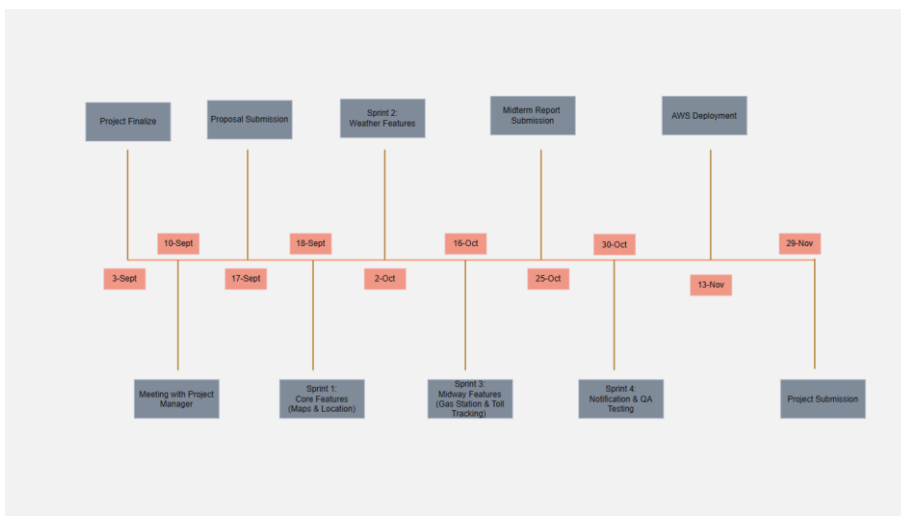


Figure 2: Milestone Chart

Gantt Charts

A Gantt chart is a project management tool that shows tasks along with their start and end dates in a timeline format. It helps the team track progress, manage deadlines, and keep the project on schedule.

In this project, the Gantt chart displays the full timeline, including each sprint, its main tasks, and duration. The work begins with Sprint 0 (Initiation) from September 3 to September 17, 2025, where the team sets up the scope, objectives, roles, and tools. After that, development follows the Agile Scrum approach with 2-week sprints.

The sprint -1 starts on September 18, 2025, and the process continues until Sprint 5, which ends on November 26, 2025. Each sprint runs for 14 days, focusing on specific tasks such as UI design, API integration, testing, and deployment. This planned approach makes it easier to track work, meet deadlines, and stay aligned with project goals and stakeholder expectations.

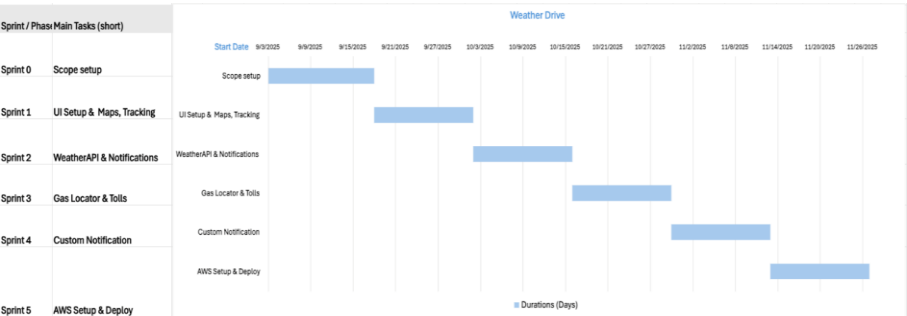


Figure 3: Gantt Chart

Implemented Feature 1: User Authentication and Onboarding System

Overview

The authentication system is the foundation of the Weather Driver application, providing secure user registration, login, and password management capabilities. This feature ensures that user preferences and data are protected while providing a seamless onboarding experience.

Implementation Details

While implementing the feature, user authentication, and onboarding, we have to go through and develop many screens. The details of work done on those screens are described below: -

1. Splash Screen (SplashScreen.js)

The application opens with a dedicated loading screen featuring the Weather Driver branding, designed to immediately establish the app's identity and create a smooth transition into the user experience. The logo display incorporates smooth animations to make the loading experience feel dynamic and professional. Once the loading phase is complete, the application transitions smoothly into the onboarding sequence or the main interface, depending on whether the user is launching the app for the first time. New users are directed to the onboarding screens and signup screen, while returning users are taken directly to the main home screen.

Screenshots



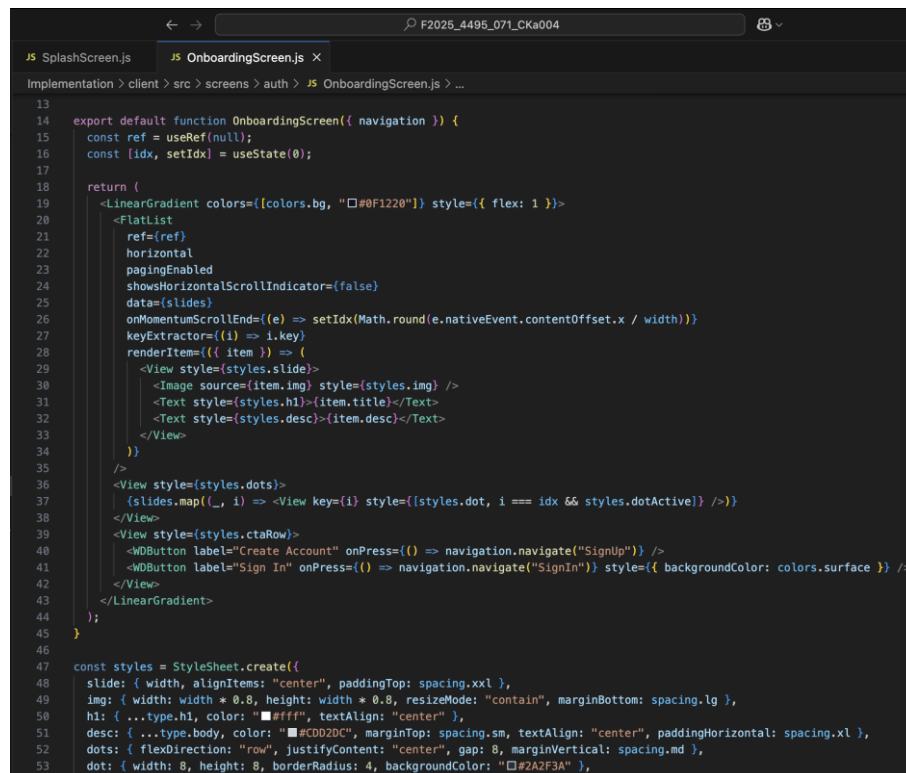
```
JS SplashScreen.js x
Implementation > client > src > screens > auth > JS SplashScreen.js > ...

6
7 export default function SplashScreen({ navigation }) {
8   return (
9     <SafeAreaView style={styles.root} edges={['top', 'bottom']}>
10       <View style={styles.container}>
11         {/* Logo Section */}
12         <View style={styles.logoContainer}>
13           {/* Cloud Car Icon - Much smaller */}
14           <Image
15             source={require('../../assets/cloudlogo.png')}
16             style={styles.icon}
17             resizeMode="contain"
18           />
19
20           {/* Weather Driver Text - Smaller */}
21           <Image
22             source={require('../../assets/letterlogo.png')}
23             style={styles.logoText}
24             resizeMode="contain"
25           />
26         </View>
27
28         {/* Buttons */}
29         <View style={styles.buttonContainer}>
30           <WDButton
31             label="Sign In"
32             onPress={() => navigation.navigate("SignIn")}
33             style={styles.signInBtn}
34             textStyle={styles.signInBtnText}
35           />
36           <WDButton
37             label="Create account"
38             onPress={() => navigation.navigate("SignUp")}
39             style={styles.createAccountBtn}
40             textStyle={styles.createAccountBtnText}
41           />
42         </View>
43       </View>
44     </SafeAreaView>
45   );
46 }
```

2. Onboarding Screen (OnboardingScreen.js)

It's a screen that welcomes users and offers a multi-step introduction to its features, which provides users with a guided walkthrough when they first open the app. Each step focuses on a specific core functionality. To enhance understanding and engagement, the introduction includes visual demonstrations of key functionalities. The interface is designed to be swipeable with progress indicators, creating a smooth and interactive user experience. Users can easily swipe left or right to move through the onboarding steps, making the process feel natural and dynamic.

Screenshots

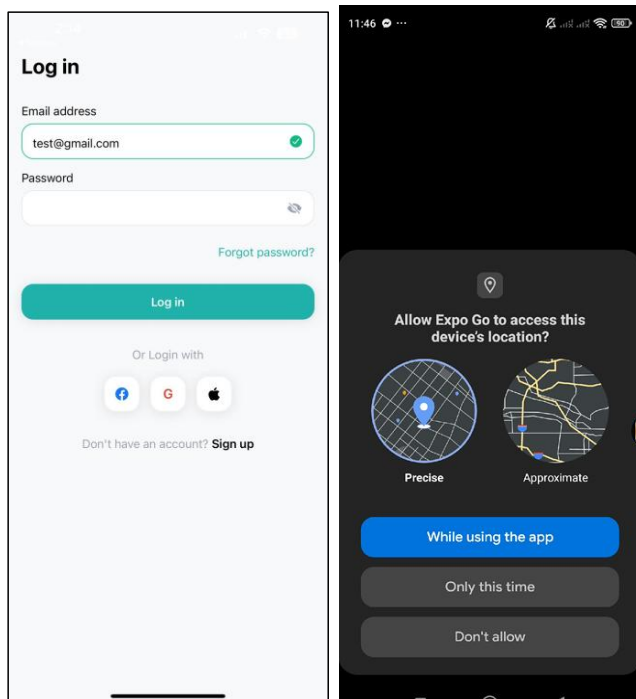


```
13
14 export default function OnboardingScreen({ navigation }) {
15   const ref = useRef(null);
16   const [idx, setIdx] = useState(0);
17
18   return (
19     <LinearGradient colors={[colors.bg, "#0F1220"]} style={{ flex: 1 }}>
20       <FlatList
21         ref={ref}
22         horizontal
23         pagingEnabled
24         showsHorizontalScrollIndicator={false}
25         data={slides}
26         onMomentumScrollEnd={e => setIdx(Math.round(e.nativeEvent.contentOffset.x / width))}
27         keyExtractor={i => i.key}
28         renderItem={({ item }) => (
29           <View style={styles.slide}>
30             <Image source={item.img} style={styles.img} />
31             <Text style={styles.h1}>{item.title}</Text>
32             <Text style={styles.desc}>{item.desc}</Text>
33           </View>
34         )}
35       />
36       <View style={styles.dots}>
37         {slides.map((_, i) => <View key={i} style={i === idx && styles.dotActive} />)}
38       </View>
39       <View style={styles.ctaRow}>
40         <MDBButton label="Create Account" onPress={() => navigation.navigate("SignUp")} />
41         <MDBButton label="Sign In" onPress={() => navigation.navigate("SignIn")} style={{ backgroundColor: colors.surface }} />
42       </View>
43     </LinearGradient>
44   );
45 }
46
47 const styles = StyleSheet.create({
48   slide: { width, alignItems: "center", paddingTop: spacing.xxl },
49   img: { width: width * 0.8, height: width * 0.8, resizeMode: "contain", marginBottom: spacing.lg },
50   h1: { ...type.h1, color: "#ffff", textAlign: "center" },
51   desc: { ...type.body, color: "#CDD2DC", marginTop: spacing.sm, textAlign: "center", paddingHorizontal: spacing.xl },
52   dots: { flexDirection: "row", justifyContent: "center", gap: 8, marginVertical: spacing.md },
53   dot: { width: 8, height: 8, borderRadius: 4, backgroundColor: "#2A2F3A" },
54 }
```

3. Sign In Screen (SignInScreen.js)

The login interface features email and password input fields with built-in validation, ensuring users enter correct and complete information before proceeding. The email field checks for a valid email format, while the password field enforces required criteria such as minimum length and non-empty input. Users are also provided with a “Remember Me” option, allowing them to stay logged in on their device for convenience during future sessions. In case a user forgets their credentials, a “Forgot Password” link is available, directing them to a secure password recovery flow to reset their login details. Additionally, new users who do not yet have an account can easily navigate to the sign-up screen through a dedicated link, encouraging smooth onboarding. The form includes real-time validation with visible error states, such as highlighting incorrect fields and displaying brief error messages to guide users toward correcting their input, creating a user-friendly and secure authentication experience.

Screenshots



```

7
8 export default function SignInScreen({ navigation }) {
9   const [email, setEmail] = useState("");
10  const [password, setPassword] = useState("");
11  const handleSocialLogin = (provider) => {
12    alert(`${provider} login clicked! (Not implemented yet)`);
13  };
14  const isEmailValid = email.includes("@") && email.includes(".");
15  return (
16    <View style={styles.root}>
17      <Text style={styles.h1}>Log in</Text>
18      <WInput
19        label="Email address"
20        placeholder="helloworld@gmail.com"
21        value={email}
22        onChangeText={setEmail}
23        keyboardType="email-address"
24        showValidation={email.length > 0}
25        isValid={isEmailValid}
26        style={styles.inputField}
27      />
28      <WInput
29        label="Password"
30        placeholder="*****"
31        value={password}
32        onChangeText={setPassword}
33        secureTextEntry={true}
34        showPasswordToggle={true}
35        style={styles.inputField}
36      />
37      <Pressable onPress={() => navigation.navigate("ForgotPassword")}>
38        <Text style={styles.forgotPassword}>Forgot password?</Text>
39      </Pressable>
40      <WDButton
41        label="Log in"
42        onPress={() => navigation.replace("Main")}
43        style={styles.loginBtn}
44      />
45      <Text style={styles.orText}>Or Login with</Text>
46      <View style={styles.socialContainer}>
47        <Pressable style={styles.socialBtn} onPress={() => handleSocialLogin("Facebook")}>
48          <Ionicons name="logo-facebook" size={20} color="#1877F2" />

```

4. Sign Up Screen (SignUpScreen.js)

The sign-up screen form provides a clear and structured way for new users to create an account by entering their essential information. It includes fields for email, password, password confirmation, and other required details, all designed to ensure accuracy and security. Each input is supported by field validations; the email field checks for a valid format, while the password field enforces strength requirements such as minimum length and character variety. Users must also confirm their password to prevent typing errors, with the system verifying that both entries match before submission. To enhance user experience, the form provides real-time validation feedback, instantly highlighting errors or confirming correct entries as users type.

This responsive interaction helps prevent submission errors and creates a smooth, intuitive, and secure registration experience.

Screenshots

12:45 100%

Sign up

Email

test@gmail.com

Create a password

Confirm password

Sign up

Or Register with

Already have an account? **Log in**

```

8  export default function SignUpScreen({ navigation }) {
9    const [email, setEmail] = useState("");
10   const [password, setPassword] = useState("");
11   const [confirmPassword, setConfirmPassword] = useState("");
12   const handleSocialLogin = (provider) => {
13     alert(`${provider} login clicked! (Not implemented yet)`);
14   };
15   const isEmailValid = email.includes("@") && email.includes(".");
16   const isPasswordValid = password.length >= 8;
17   const doPasswordsMatch = password === confirmPassword && confirmPassword.length > 0;
18   return (
19     <View style={styles.root}>
20       <Text style={styles.h1}>Sign up</Text>
21       <WDInput
22         label="Email"
23         placeholder="example@gmail.com"
24         value={email}
25         onChangeText={setEmail}
26         keyboardType="email-address"
27         showValidation={email.length > 0}
28         isValid={isEmailValid}
29         style={styles.inputField}
30       />
31       <WDInput
32         label="Create a password"
33         placeholder="must be 8 characters"
34         value={password}
35         onChangeText={setPassword}
36         secureTextEntry={true}
37         showPasswordToggle={true}
38         showValidation={password.length > 0}
39         isValid={isPasswordValid}
40         style={styles.inputField}
41       />
42       <WDInput
43         label="Confirm password"
44         placeholder="repeat password"
45         value={confirmPassword}
46         onChangeText={setConfirmPassword}
47         secureTextEntry={true}

```

5. Forgot Password Screen (ForgotPasswordScreen.js)

The Forgot Password screen provides a simple and secure way for users to reset their passwords when they forget their login credentials. It features a single email input field where users can enter the email address associated with their account. Before proceeding, the system performs input validation to ensure that the entered email is not empty and follows the correct format. This prevents invalid or incomplete submissions and helps maintain account security. Once the email is validated, the app initiates the password recovery process by sending a reset code or link to the user's registered email address. This approach ensures that only verified users can

proceed with resetting their passwords, offering both convenience and protection in the password recovery flow.

Screenshots

<

Forgot password?

Don't worry! It happens. Please enter the email associated with your account.

Email address

Send code

Remember password? [Log in](#)

```

← → F2025_4495_071_CKa004
JS ForgotPasswordScreen.js •
Implementation > client > src > screens > auth > JS ForgotPasswordScreen.js > ForgotPasswordScreen
8 export default function ForgotPasswordScreen({ navigation }) {
9   const [email, setEmail] = useState("");
10  const handleSendCode = () => {
11    if (email.trim()) {
12      navigation.navigate("ForgotPasswordCode", { email });
13    }
14  };
15  return (
16    <View style={styles.root}>
17      <Pressable onPress={() => navigation.goBack()} style={styles.backBtn}>
18        <Icons name="chevron-back" size={24} color={colors.text} />
19      </Pressable>
20      <Text style={styles.h1}>Forgot password?</Text>
21      <Text style={styles.desc}>
22        Don't worry! It happens. Please enter the email associated with your account.
23      </Text>
24      <WInput
25        label="Email address"
26        placeholder="Enter your email address"
27        value={email}
28        onChangeText={setEmail}
29        keyboardType="email-address"
30        style={styles.inputField}
31      />
32      <WDButton
33        label="Send code"
34        onPress={handleSendCode}
35        style={styles.sendBtn}
36      />
37      <Pressable onPress={() => navigation.navigate("SignIn")}>
38        <Text style={styles.remember}>
39          Remember password? <Text style={styles.loginLink}>Log in</Text>
40        </Text>
41      </Pressable>
42    </View>
43  );
44 }
45
46 const styles = StyleSheet.create({
47   root: {
48     flex: 1,
49     backgroundColor: colors.surface,

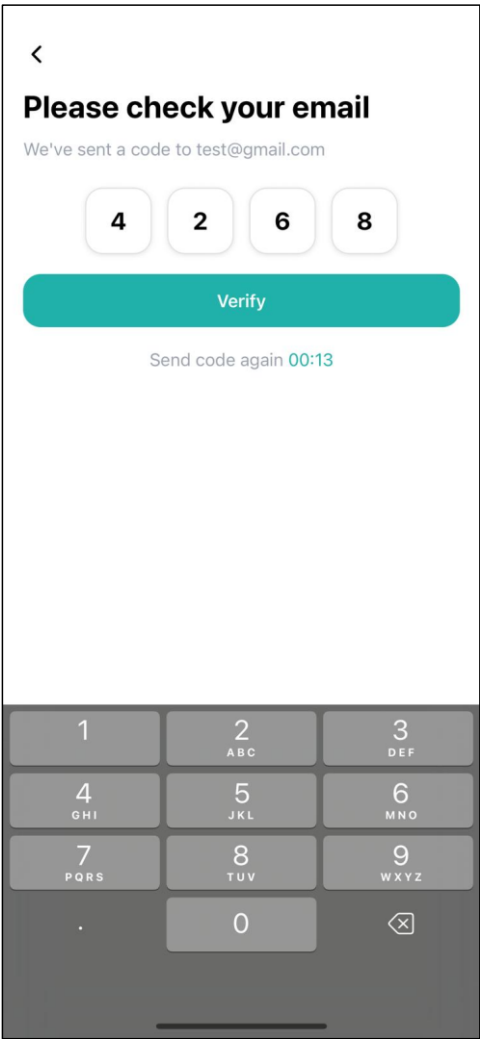
```

6. Forgot Password Code Screen (ForgotPasswordCodeScreen.js)

The Forgot Password Code screen allows users to verify their identity by entering the verification code sent to their registered email during the password reset process. The screen includes a dedicated input field for users to enter this code, ensuring a secure and straightforward verification step. The application performs code validation to check that the entered code matches the one sent and that it hasn't expired, preventing unauthorized or

incorrect attempts. If users do not receive the code or if it expires, the screen provides a “Resend Code” option, allowing them to request a new verification code quickly. This feature enhances both security and user convenience, ensuring that the password recovery process remains smooth, reliable, and user-friendly.

Screenshots



```

< -- F2025_4495_071_Cka004

JS ForgotPasswordCodeScreen.js
Implementation > client > src > screens > auth > JS ForgotPasswordCodeScreen.js > ForgotPasswordCodeScreen > [e]

7 export default function ForgotPasswordCodeScreen({ navigation, route }) {
8   const { email } = route.params || {};
9   const [code, setCode] = useState("");
10  const [timer, setTimer] = useState(20);
11  const inputRef = useRef(null);
12  useEffect(() => {
13    const interval = setInterval(() => {
14      setTimer((prev) => (prev > 0 ? prev - 1 : 0));
15    }, 1000);
16    return () => clearInterval(interval);
17  }, []);
18  const handleVerify = () => {
19    if (code.length === 4) {
20      navigation.navigate("ResetPassword", { email, code });
21    }
22  };
23  const handleResendCode = () => {
24    setTimer(20);
25    setCode("");
26  };
27  const displayCode = () => {
28    const digits = code.split("");
29    const result = ["", "", "", ""];
30    for (let i = 0; i < digits.length && i < 4; i++) {
31      result[i] = digits[i];
32    }
33    return result;
34  };
35  return (
36    <View style={styles.root}>
37      <Pressable onPress={() => navigation.goBack()} style={styles.backBtn}>
38        <Ionicons name="chevron-back" size={24} color="#000" />
39      </Pressable>
40      <Text style={styles.h1}>Please check your email</Text>
41      <Text style={styles.desc}>
42        We've sent a code to {email}
43      </Text>
44      <View style={styles.codeContainer}>
45        {displayCode().map((digit, index) => (
46          <Pressable
47            key={index}

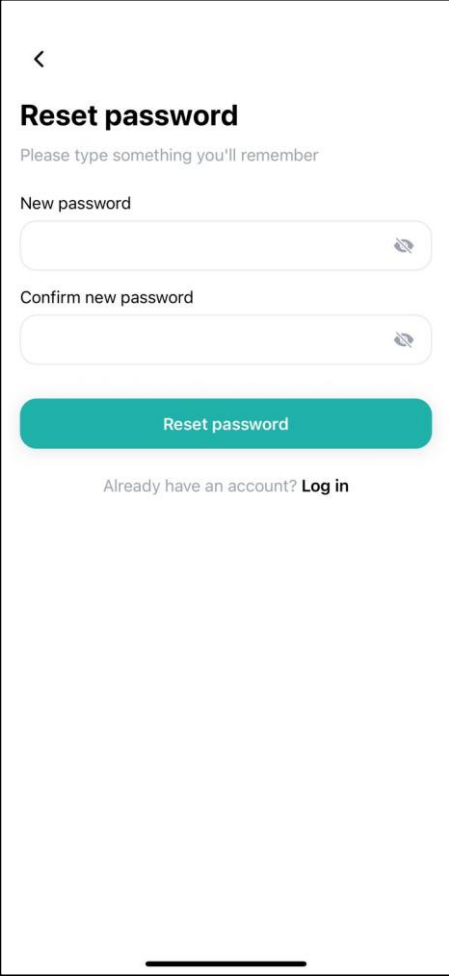
```

7. Reset Password Screen (ResetPasswordScreen.js)

The Reset Password screen enables users to securely create a new password after successfully verifying their identity. It includes input fields for both the new password and its confirmation, ensuring that users enter the same password twice to prevent typing mistakes. The screen clearly displays password strength requirements, such as minimum length, inclusion of numbers, symbols, or uppercase letters, helping users create a strong and secure password. As users type, real-time indicators may show whether these requirements are met. Additionally, the form includes validation checks consistent with the original sign-up criteria, ensuring that

the new password aligns with the app’s security standards. This consistent and guided approach provides a seamless, safe, and user-friendly experience during the password reset process.

Screenshots



<

Reset password

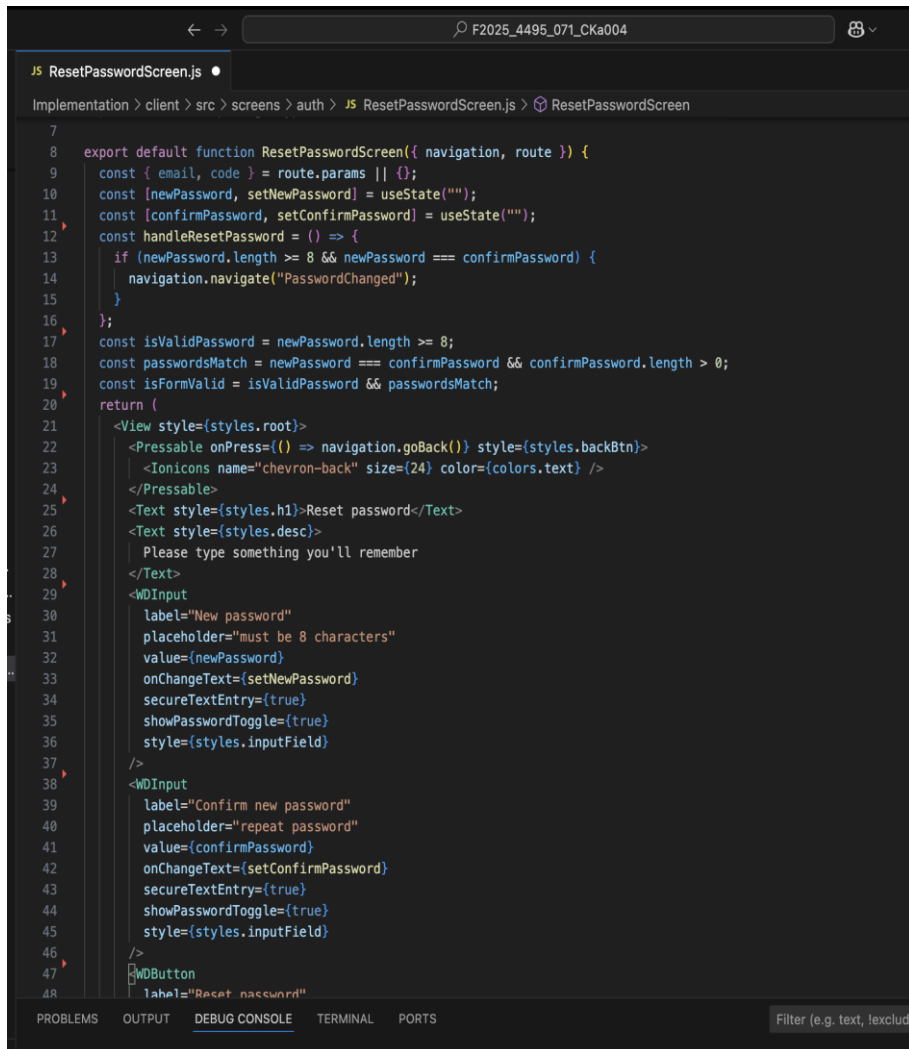
Please type something you'll remember

New password

Confirm new password

Reset password

Already have an account? [Log in](#)



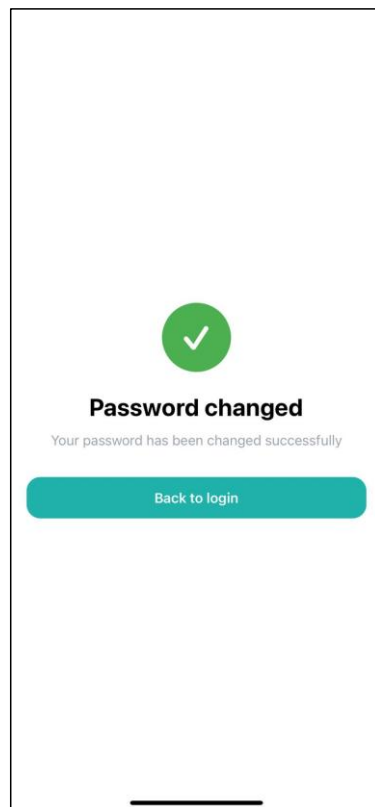
```
7
8 export default function ResetPasswordScreen({ navigation, route }) {
9   const { email, code } = route.params || {};
10  const [newPassword, setNewPassword] = useState("");
11  const [confirmPassword, setConfirmPassword] = useState("");
12  const handleResetPassword = () => {
13    if (newPassword.length >= 8 && newPassword === confirmPassword) {
14      navigation.navigate("PasswordChanged");
15    }
16  };
17  const isValidPassword = newPassword.length >= 8;
18  const passwordsMatch = newPassword === confirmPassword && confirmPassword.length > 0;
19  const isValidForm = isValidPassword && passwordsMatch;
20  return (
21    <View style={styles.root}>
22      <Pressable onPress={() => navigation.goBack()} style={styles.backBtn}>
23        <Ionicons name="chevron-back" size={24} color={colors.text} />
24      </Pressable>
25      <Text style={styles.h1}>Reset password</Text>
26      <Text style={styles.desc}>
27        Please type something you'll remember
28      </Text>
29      <WDInput
30        label="New password"
31        placeholder="must be 8 characters"
32        value={newPassword}
33        onChangeText={setNewPassword}
34        secureTextEntry={true}
35        showPasswordToggle={true}
36        style={styles.inputField}
37      />
38      <WDInput
39        label="Confirm new password"
40        placeholder="repeat password"
41        value={confirmPassword}
42        onChangeText={setConfirmPassword}
43        secureTextEntry={true}
44        showPasswordToggle={true}
45        style={styles.inputField}
46      />
47      <WDButton
48        label="Reset password"
49      />
50    </View>
51  );
52}
```

8. Password Changed Screen (PasswordChangedScreen.js)

The Password Changed screen (PasswordChangedScreen.js) serves as the final confirmation step in the password recovery process, reassuring users that their password has been successfully reset. This screen typically displays a clear success message along with a visual indicator, such as a checkmark, to confirm the completion of the process. It provides users with a sense of closure and security, knowing their account credentials have been updated. From this screen, users can easily navigate back to the sign-in page, where they can log in using their

new password. This smooth transition ensures a seamless user experience and effectively concludes the password reset workflow.

Screenshots:



```

F2025_4495_071_CKa004

JS PasswordChangedScreen.js
Implementation > client > src > screens > auth > JS PasswordChangedScreen.js > PasswordChangedScreen.js
1  import React from "react";
2  import { View, Text, StyleSheet } from "react-native";
3  import WDButton from "../../components/ui/WDButton";
4  import { colors, spacing, type } from "../../theme/tokens";
5
6  export default function PasswordChangedScreen({ navigation }) {
7    const handleBackToLogin = () => {
8      navigation.navigate("SignIn");
9    };
10   return (
11     <View style={styles.root}>
12       <View style={styles.content}>
13         <View style={styles.successContainer}>
14           <View style={styles.checkmarkCircle}>
15             <Text style={styles.checkmark}></Text>
16           </View>
17         </View>
18
19         <Text style={styles.h1}>Password changed</Text>
20         <Text style={styles.desc}>
21           Your password has been changed successfully
22         </Text>
23
24         <WDButton
25           label="Back to login"
26           onPress={handleBackToLogin}
27           style={styles.loginBtn}
28         </WDButton>
29       </View>
30     </View>
31   );
32 }
33 const styles = StyleSheet.create({
34   root: {
35     flex: 1,
36     backgroundColor: "#fff",
37     padding: spacing.lg
38   },
39   content: {
40     flex: 1,
41     alignItems: "center",
42     justifyContent: "center"

```

Technical Implementation

Validation System

The validation system is implemented as a modular client-side layer that runs on every auth-related form. Email inputs are validated with well-tested regex patterns to ensure proper structure (local-part, “@”, domain, and TLD) before submission. Passwords must meet defined strength requirements. It is evaluated in real time with a strength meter and checklist. All required fields are enforced at both the UI and schema level, surfacing clear, contextual error messages (e.g., “Please enter a valid email” or “Password must include a special character”) and preventing submission until issues are resolved.

UI Components

The UI layer is built around a small, consistent design system. A reusable input component (WDInput.js) standardizes typography, spacing, focus rings, and validation states across forms, exposing props like label, placeholder for inline guidance; it also includes accessibility labels, proper hit targets, and error/valid icons for clarity. The custom button (WDButton.js) supports primary/secondary variants, size options, and built-in loading states (spinner + disabled interaction) to prevent double-submits and communicate progress. A lightweight Logo.js component centralizes the Weather Driver logomark/wordmark, ensuring brand consistency across splash, auth, and header contexts. All components integrate error state handling by design: invalid fields surface friendly, actionable messages (“Enter a valid email”), and align with the same color tokens and accessible experience that’s easy to theme and test.

Navigation Flow

It begins with the Splash screen, which displays the Weather Driver branding and logo, and then transitions into the Onboarding sequence, where users are introduced to the app’s key features through swipe-controlled tutorials. From there, users proceed to the Sign In screen, with the option to switch to the Sign-Up screen if they are new to the platform. In cases where users forget their password, they can navigate from the Sign In screen to the Forgot Password flow, which sequentially guides them through several steps — entering their email, verifying the code on the Code Entry screen, creating a new password on the Reset Password screen, and finally reaching the Password Changed confirmation screen. Once the reset process is complete, users are smoothly directed back to the Sign In screen to access their account with the newly updated credentials. This structured navigation ensures a user-friendly experience, minimizing confusion and maintaining consistency throughout the authentication process.

Splash → Onboarding → Sign In ⇔ Sign Up. (If forget the password) → Forgot Password → Code Entry → Reset Password → Password Changed → Sign In.

Challenges and Solutions

Challenge 1: TypeScript to JavaScript Conversion

Issue: The provided codebase used TypeScript, unfamiliar to the team

Solution: Client agreed to JavaScript implementation after meeting on Sept 24, 2025

Impact: Allowed the team to focus on functionality rather than language learning

Challenge 2: Form Validation Consistency

Issue: Ensuring validation rules match across sign-up and password reset

Solution: Created centralized validation functions

Implementation: Collaborative work on Oct 4, 2025 (Chetan, Pukar, Pravesh)

Challenge 3: Expo Configuration Issues

Issue: Initial Expo build errors blocking progress.

Solution: Package version conflicts resolved using Stack Overflow research

Result: Successful build achieved.

Testing Performed

The testing phase focused on ensuring that every component of the authentication system functions smoothly and reliably across various scenarios. Manual testing was conducted on all input fields to verify accurate data entry, field responsiveness, and proper handling of user interactions. Each form underwent validation error message testing to confirm that incorrect inputs triggered clear, context-specific error prompts, ensuring users receive immediate and helpful feedback. The entire navigation flow from the splash and onboarding screens to sign-in, sign-up, and password recovery was thoroughly tested to confirm that transitions between screens occurred seamlessly and without navigation loops or dead ends. Special attention was given to screen transition animations, checking for smoothness and timing consistency across devices. Error state handling was validated to ensure that the application displayed appropriate messages during invalid submissions or failed network requests. Lastly, form submission behavior was tested to confirm successful data handling, proper redirects, and accurate feedback after each user action. Overall, the testing ensured a stable, user-friendly, and visually consistent authentication experience.

Summary

The authentication screens are designed in full alignment with the Figma design specifications, ensuring a cohesive and visually appealing user experience across the application. Each screen maintains a consistent color scheme derived from the app's defined design tokens, providing uniformity in elements such as buttons, input fields, and background gradients.

Implemented Feature 2: Core Navigation and Map Interface

Overview

The navigation system provides the structural foundation for the application, implementing a bottom tab navigator with four main sections: Map, Alerts, Search, and Settings. The map interface serves as the primary screen where users interact with location-based weather data.

Implementation Details

Main Navigation Structure (App.js):

The navigation structure was implemented using React Navigation, configured through a bottom-tab navigator that links four main screens. The design follows the Figma guidelines for color, layout, and iconography, incorporating icons for visually distinct tab icons with active and inactive states. The transitions between screens are smooth, ensuring consistent experience across devices.

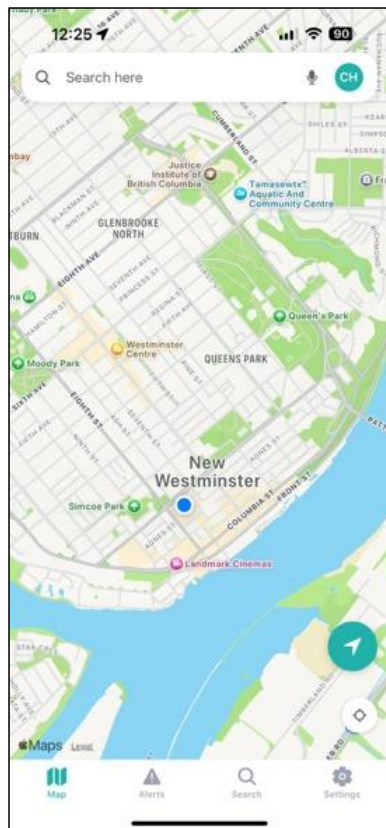
```
App.js
Implementation > client > JS App.js > App
28 function MainTabs() {
29   return (
30     <Tab.Navigator
31       screenOptions={({ route }) => ({
32         headerShown: false,
33         tabBarStyle: { backgroundColor: colors.surface, borderTopColor: colors.border },
34         tabBarActiveTintColor: colors.primary,
35         tabBarInactiveTintColor: colors.muted,
36         tabBarIcon: ({ color, size }) => {
37           const n = { Map: "map", Alerts: "warning", Search: "search", Settings: "settings" };
38           return <Ionicons name={n[route.name]} size={size} color={color} />;
39         }
40       }) >
41     <Tab.Screen name="Map" component={MapScreen} />
42     <Tab.Screen name="Alerts" component={AlertsScreen} />
43     <Tab.Screen name="Search" component={SearchScreen} />
44     <Tab.Screen name="Settings" component={SettingMenuScreen} />
45   </Tab.Navigator>
46 );
47
48 const navTheme = {
49   ...DefaultTheme,
50   colors: {
51     ...DefaultTheme.colors, background: colors.bg, text: colors.text, border: colors.border, card: colors.surface
52   }
53 };
54
55 export default function App() {
56   return (
57     <QueryClientProvider client={queryClient}>
58       <NavigationContainer theme={navTheme}>
59         <StatusBar style="light" />
60         <RootStack.Navigator screenOptions={({ headerShown: false })>
61           <RootStack.Screen name="Splash" component={SplashScreen} />
62           <RootStack.Screen name="Onboarding" component={OnboardingScreen} />
63           <RootStack.Screen name="Signup" component={SignupScreen} />
64           <RootStack.Screen name="ForgotPassword" component={ForgotPasswordScreen} />
65           <RootStack.Screen name="ForgotPasswordCode" component={ForgotPasswordCodeScreen} />
66           <RootStack.Screen name="ResetPassword" component={ResetPasswordScreen} />
67           <RootStack.Screen name="PasswordChanged" component={PasswordChangedScreen} />
68           <RootStack.Screen name="Main" component={MainTabs} />
69           <RootStack.Screen name="GasStation" component={GasStationScreen} />
70           <RootStack.Screen name="PreferredBrand" component={PreferredBrandScreen} />
71         </RootStack.Navigator>
72       </NavigationContainer>
73     </QueryClientProvider>
74   );
75 }
```

Tab Screens:

1. Map Screen (MapScreen.js)

The Map Screen serves as the home interface and was built using React Native Maps. It includes a responsive search bar, an integrated profile icon, and a set of quick-access chips at the bottom for frequently used features. The screen displays the user's current location using the Expo Location API and the current phone GPS location. Also, placeholders have been implemented for future weather overlays and data layers.

Screenshots



```
← → F2025_4495_071_CKa004

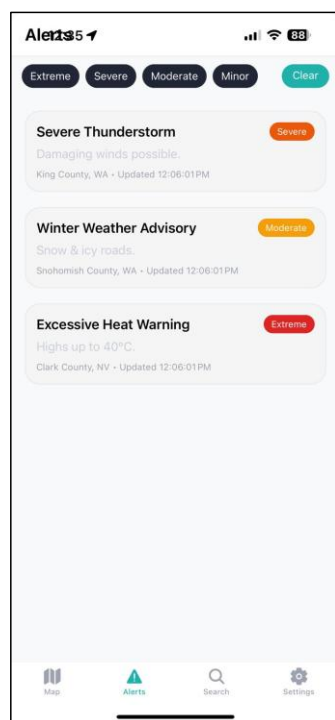
JS MapScreen.js •
Implementation > client > src > screens > main > JS MapScreen.js > MapScreen

12 export default function MapScreen({ navigation }) {
13   const [region, setRegion] = useState(null);
14   const [showProfileMenu, setShowProfileMenu] = useState(false);
15   const [searchText, setSearchText] = useState("");
16   const [showFilters, setShowFilters] = useState(false);
17   const [showRouteModal, setShowRouteModal] = useState(false);
18   const mapRef = useRef(null);
19   useEffect(() => {
20     (async () => {
21       const { status } = await Location.requestForegroundPermissionsAsync();
22       if (status !== "granted") return;
23       const loc = await Location.getCurrentPositionAsync({});
24       setRegion({
25         latitude: loc.coords.latitude,
26         longitude: loc.coords.longitude,
27         latitudeDelta: 0.25,
28         longitudeDelta: 0.25
29       });
30     })();
31   }, []);
32   const handleProfileMenuPress = (option) => {
33     setShowProfileMenu(false);
34     if (option === "Logout") {
35       navigation.replace("SignIn");
36     } else if (option === "Plan a drive") {
37       setShowRouteModal(true);
38     } else if (option === "Settings") {
39       navigation.navigate("Settings");
40     }
41     // Handle other menu options
42   };
43   const handleMarkerPress = (alert) => {
44     // Show route when user taps on a marker
45     setShowRouteModal(true);
46   };
47   return (
48     <View style={styles.container}>
49       {region ? (
50         <MapView
51           ref={mapRef}
52           style={StyleSheet.absoluteFill}
```

2. Alerts Screen (AlertScreen.js)

The Alerts Screen displays categorized hazard notifications using reusable card and badge components. Although the backend integration for live alerts has not yet been implemented, the UI layout provides the hint of the future UI which of real-time updates once connected. The Search Screen allows users to locate specific destinations and view recent queries through a clean and minimal interface, while the Settings Menu Screen acts as a gateway to additional sub-pages such as Preferences, Speedometer, and Gas Station modules. Each tab maintains consistent spacing, color schemes, and typography, ensuring design uniformity throughout the app.

Screenshot:



```

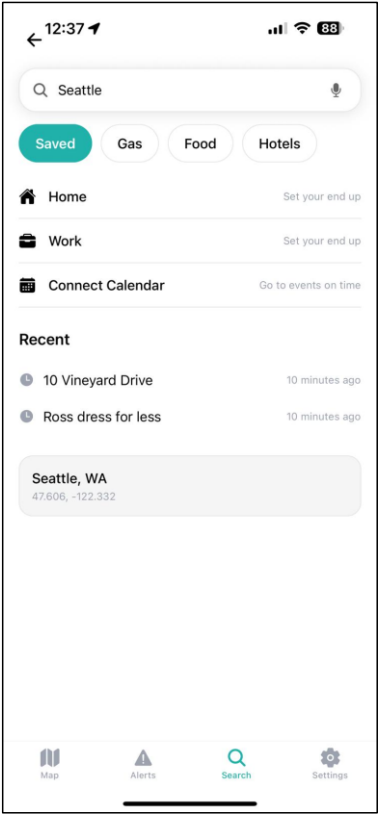
← → F2025_4495_071_CKa004
JS AlertScreen.js
Implementation > client > src > screens > main > JS AlertScreen.js > AlertsScreen
7  const SAMPLE_ALERTS = [
9    { id: "1", title: "Winter Weather Advisory", description: "Snow & icy roads.", severity: "Moderate", area: "C" },
10   { id: "3", title: "Excessive Heat Warning", description: "Highs up to 40°C.", severity: "Extreme", area: "C" },
11 ];
12 export default function AlertsScreen() {
13   const [severity, setSeverity] = useState("");
14   const data = useMemo(
15     () => (severity ? SAMPLE_ALERTS.filter(a => a.severity === severity) : SAMPLE_ALERTS),
16     [severity] );
17   return (
18     <View style={{ flex: 1, backgroundColor: colors.bg }}>
19       <AppBar title="Alerts" />
20       <View style={styles.filterRow}>
21         {SEVERITIES.map(s => (
22           <Pressable key={s} onPress={() => setSeverity(severity === s ? "" : s)}
23             style={styles.chip, severity === s && styles.chipActive}>
24             <Text style={styles.chipText}>{s}</Text></Pressable> ))}
25         <Pressable onPress={() => setSeverity("")} style={styles.clearBtn}>
26           <Text style={{ color: "#fff" }}>Clear</Text>
27         </Pressable>
28       </View>
29       <FlatList
30         data={data}
31         keyExtractor={({i}) => i.id}
32         renderItem={({ item }) => <AlertCard {...item} />}
33         contentContainerStyle={{ paddingVertical: spacing.sm }} />
34     </View>);
35   const styles = StyleSheet.create({
36     filterRow: { flexDirection: "row", flexWrap: "wrap", gap: 8, paddingHorizontal: spacing.md, paddingVertical: spacing.md },
37     chip: { paddingVertical: 6, paddingHorizontal: 10, borderRadius: radius.pill, backgroundColor: "#212636", margin: 2 },
38     chipActive: { backgroundColor: "#3B3F52" },
39     chipText: { color: "#fff" },
40     clearBtn: { marginLeft: "auto", paddingHorizontal: spacing.md, paddingVertical: 6, borderRadius: radius.md, margin: 2 },
41   });

```

3. Search Screen (SearchScreen.js)

The Search Screen (SearchScreen.js) provides users with an interactive and efficient way to explore locations and destinations within the Weather Driver application. It includes a location search functionality that allows users to type in specific place names, addresses, or landmarks, with instant suggestions appearing as they type. This feature should be powered by an integrated API, which is not fully developed yet. We only have the frontend UI Search Screen. The screen also features a recent searches display, enabling users to quickly revisit previously searched locations without having to retype them, thereby enhancing convenience and user experience.

Screenshots:



```

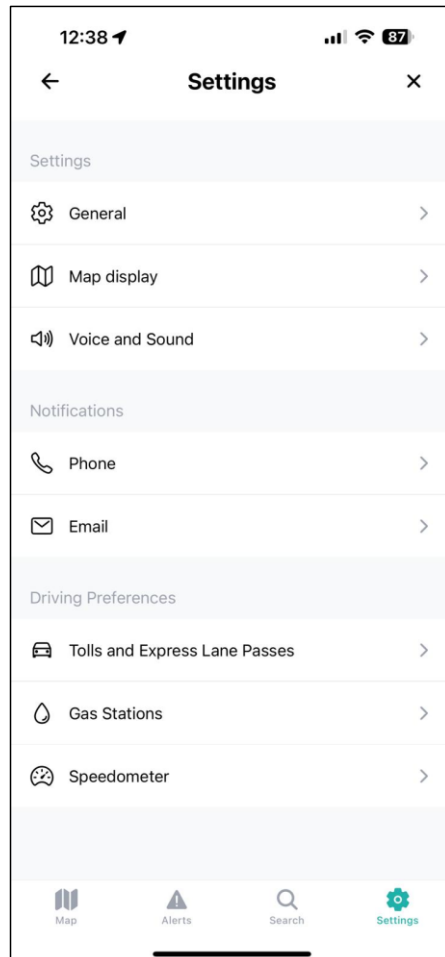
JS SearchScreen.js
Implementation > client > src > screens > main > JS SearchScreen.js > SearchScreen
1  import React, { useState } from "react";
2  import { View, Text, TextInput, StyleSheet, FlatList, Pressable, ScrollView } from "react-native";
3  import { Ionicons } from "@expo/vector-icons";
4  import { colors, spacing, radius, type, shadows } from "../../theme/tokens";
5
6  export default function SearchScreen({ navigation }) {
7    const [q, setQ] = useState("");
8    const [activeTab, setActiveTab] = useState("Saved");
9    const recentSearches = [
10      "10 Vineyard Drive",
11      "Ross dress for less"
12    ];
13    const tabs = ["Saved", "Gas", "Food", "Hotels"];
14    const results = q ? [{ name: "Seattle, WA", lat: 47.6062, lon: -122.3321 }] : [];
15    return (
16      <View style={styles.container}>
17        </* Header */>
18        <View style={styles.header}>
19          <Pressable onPress={() => navigation.goBack()} style={styles.backButton}>
20            <Ionicons name="arrow-back" size={24} color={colors.text} />
21          </Pressable>
22        </View>
23        </* Search Bar */>
24        <View style={styles.searchContainer}>
25          <View style={styles.searchBar}>
26            <Ionicons name="search" size={20} color="#666" style={styles.searchIcon} />
27            <TextInput
28              style={styles.searchInput}
29              placeholder="Search here"
30              placeholderTextColor="#666"
31              value={q}
32              onChangeText={setQ}
33              autoFocus={true}
34            />

```

4. Settings Menu Screen (SettingsMenuScreen.js)

The Settings Menu Screen serves as the central hub for user preferences and application configurations within the Weather Driver app. It offers a comprehensive organization of settings. Each category is designed with intuitive icons and clear labels to help users quickly locate and adjust specific features. The screen supports navigation to sub-settings pages, allowing users to dive deeper into individual sections without cluttering the main settings view. Additionally, the Settings Menu is accessible from multiple entry points, including the bottom tab bar and the profile modal, ensuring that users can access it conveniently from anywhere within the app. This well-structured and accessible design ensures that users have full control over their app experience while maintaining a consistent and user-friendly interface throughout.

Screenshots

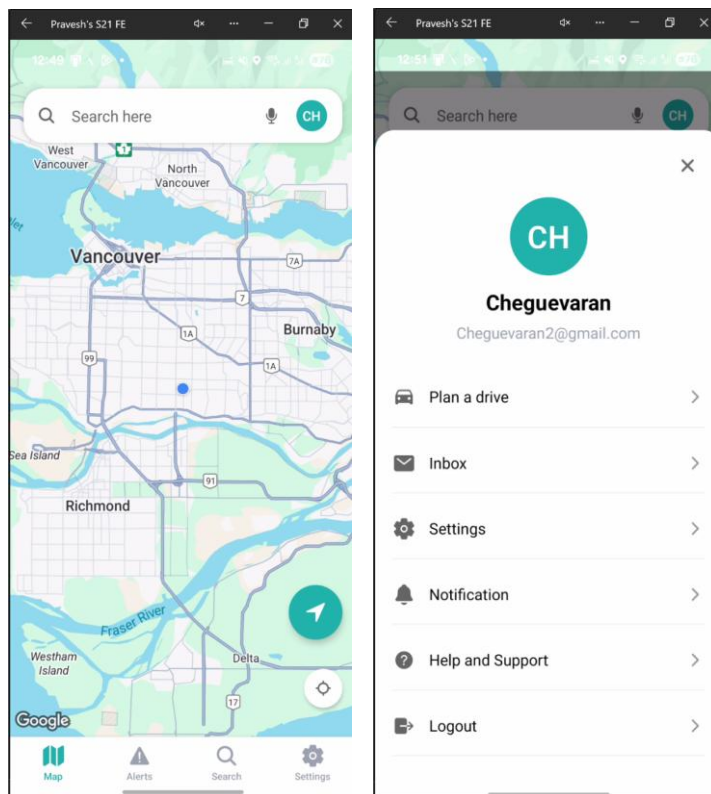


```
← → F2025_4495_071_Cka004
JS SettingsMenuScreen.js
Implementation > client > src > screens > main > JS SettingsMenuScreen.js > SettingsMenuScreen
5
6 export default function SettingsMenuScreen({ navigation }) {
7   > const settingsOptions = [
31   ];
32   return (
33     <View style={styles.container}>
34       { /* Header */ }
35       <View style={styles.header}>
36         <Pressable
37           style={styles.backButton}
38           onPress={() => navigation.goBack()}>
39           <Ionicons name="arrow-back" size={24} color={colors.text} />
40         </Pressable>
41         <Text style={styles.headerTitle}>Settings</Text>
42       <Pressable ...
47     </Pressable>
48   </View>
49   { /* Settings List */ }
50   <ScrollView style={styles.scrollView} showsVerticalScrollIndicator={false}>
51     {settingsOptions.map((section, sectionIndex) => (
52       <View key={sectionIndex} style={styles.section}>
53         {section.section && (
54           <Text style={styles.sectionHeader}>{section.section}</Text>
55         )}
56         <View style={styles.sectionItems}>
57           {section.items.map((item, itemIndex) => (
58             <Pressable
59               key={itemIndex}
60               style={[
61                 styles.settingsItem,
62                 itemIndex === section.items.length - 1 && styles.settingsItemLast
63               ]}
64               onPress={() => {
65                 // Navigate to specific settings screen
66                 if (item.route) {
67                   navigation.navigate(item.route);
68                 }
69               }}
70             >
71             <View style={styles.settingsItemLeft}>
72               <Ionicons name={item.icon} size={24} color={colors.text} />
73               <Text style={styles.settingsItemText}>{item.label}</Text>
```

Map Interface Features

The map interface was carefully constructed to deliver both functionality and an engaging visual experience. The full-screen map display provides intuitive pan and zoom capabilities, while the search bar and profile button overlay at the top keep essential functions within easy reach. The bottom-positioned action chips allow users to quickly access important features without disrupting map visibility. Additional components, such as a route-planning modal (RouteModal.js) was prepared for integration once backend support becomes available.

Real-time GPS tracking was successfully tested through the Expo Go app, and the map responsiveness was verified across various screen sizes. The design adopts the project's weather-themed aesthetic, ensuring that the map view feels cohesive with the broader visual identity of Weather Driver.



```
← → F2025_4495_071_CKa004
JS RouteModal.js • JS MapScreen.js ×
Implementation > client > src > screens > main > JS MapScreen.js > MapScreen
14 export default function MapScreen({ navigation }) {
85     /* Search Bar */
86     <View style={styles.searchContainer}>
87       <Pressable style={styles.searchBar} onPress={() => navigation.navigate('Search')}>
88         <Ionicons name="search" size={20} color="#666" style={styles.searchIcon} />
89         <View style={styles.searchContent}>
90           <Text style={styles.searchPlaceholder}>Search here</Text>
91         </View>
92         <Pressable style={styles.voiceButton}>
93           <Ionicons name="mic" size={20} color="#666" />
94         </Pressable>
95         <Pressable style={styles.profileAvatarButton} onPress={() => setShowProfileMenu(true)}>
96           <View style={styles.profileAvatar}>
97             <Text style={styles.profileAvatarText}>CH</Text>
98           </View>
99         </Pressable>
100       </Pressable>
101     </View>
102
103     /* Floating Route Button */
104     <Pressable
105       style={styles.routeButton}
106       onPress={() => setShowRouteModal(true)}
107     >
108       <Ionicons name="navigate" size={24} color="#fff" />
109     </Pressable>
110
111     /* Custom Location Button */
112     <Pressable
113       style={styles.customLocationButton}
114       onPress={async () => {
115         try {
116           const location = await Location.getCurrentPositionAsync({
117             accuracy: Location.Accuracy.High,
118           });
119         }
120       }}
119     </Pressable>
```

```
JS RouteModal.js • JS MapScreen.js X
Implementation > client > src > screens > main > JS MapScreen.js > MapScreen > <function>
14 export default function MapScreen({ navigation }) {
15
168   /* Profile Menu Modal */
169   <Modal
170     visible={showProfileMenu}
171     transparent={true}
172     animationType="slide"
173     onRequestClose={() => setShowProfileMenu(false)}
174   >
175     <View style={styles.profileModalContainer}>
176       <View style={styles.profileModal}>
177         /* Close Button */
178         <Pressable
179           style={styles.closeButton}
180           onPress={() => setShowProfileMenu(false)}
181         >
182           <Ionicons name="close" size={24} color="#666" />
183         </Pressable>
184
185         /* Profile Header */
186         <View style={styles.profileModalHeader}>
187           <View style={styles.profileImageContainer}>
188             <View style={styles.profileImage}>
189               <Text style={styles.profileImageText}>CH</Text>
190             </View>
191           </View>
192           <Text style={styles.profileModalName}>Cheguevaran</Text>
193           <Text style={styles.profileModalEmail}>Cheguevaran2@gmail.com</Text>
194         </View>
195
196         /* Menu Items */
197         <View style={styles.profileMenuItems}>
198           <Pressable style={styles.profileMenuItem} onPress={() => handleProfileMenuPress("Plan a drive")}>
199             <Ionicons name="car" size={24} color="#666" />
200             <Text style={styles.profileMenuText}>Plan a drive</Text>
201             <Ionicons name="chevron-forward" size={20} color="#999" />
202           </Pressable>
203
204           <Pressable style={styles.profileMenuItem} onPress={() => handleProfileMenuPress("Inbox")}>
205             <Ionicons name="mail" size={24} color="#666" />
206             <Text style={styles.profileMenuText}>Inbox</Text>
207             <Ionicons name="chevron-forward" size={20} color="#999" />
208           </Pressable>
209         </View>
210       </View>
211     </View>
212   </Modal>
213 }
214
```

Technical Architecture

The navigation flow is organized through a Root Stack Navigator, which separates the authentication process from the main application. The Auth Stack includes screens such as Splash, Onboarding, Sign-In, and Password Reset, while the Main Tabs handle the primary user interface. Within the main navigation system, four major tabs, Map, Alerts, Search, and Settings, serve as entry points to specific functional modules. Sub-screens, including Gas Station, Preferred Brand, and Speedometer, are nested under the Settings tab for better modularity.

State management is handled primarily through React Navigation's internal state system, ensuring that tab persistence and navigation memory are maintained between transitions. The

map's location state is controlled using the Expo Location hook, allowing the app to retrieve and display the user's real-time position. Temporary state variables were used to manage search queries and location history, while a global context for user preferences has been structured for implementation in a future sprint.

Collaboration Highlights

This feature was the product of close collaboration among team members, especially Chetan Kaur and Pravesh Poudel, who worked together on refining the map interface layout. On October 5, 2025, we collaborated to position the search bar and action chips correctly, configure the map canvas, and integrate the profile icon in accordance with the Figma design. We also optimized the navigation flow and conducted UI testing on multiple devices to ensure responsiveness. This cooperative approach enhanced both the visual consistency and usability of the navigation experience.

Summary

The Core Navigation and Map Interface feature successfully establishes the navigational backbone of the Weather Driver app. It enables fluid user transitions, integrates real-time GPS tracking, and lays the groundwork for advanced weather and alert functionalities. While the backend is not yet active, the UI/UX structure is fully implemented and ready for data integration in upcoming sprints.

Implemented Feature 3: Settings Management System

Overview

The settings system provides users with comprehensive control over application behavior, preferences, and customization options. This feature is critical for personalizing the user experience and managing application configuration.

Implementation Details

Settings Menu Screen (SettingsMenuScreen.js)

The Settings Menu Screen, implemented by Pravesh Poudel on October 7, 2025, serves as the entry point to all configuration and customization options. It is divided into three logical sections: General Settings, Notifications, and Driving Preferences, allowing users to easily navigate between different categories. General Settings include placeholders for map display and voice options, while the Notifications section contains toggles for phone and email alerts. The Driving Preferences section links directly to the Gas Station Settings and Speedometer Settings screens, which were developed as submodules. Navigation between these pages is managed using the React Navigation stack, with consistent transitions and back navigation support.

Icons were integrated using Ionicons, while spacing, typography, and color choices strictly follow the design tokens from Figma. This approach guarantees both usability and a clean aesthetic consistent with the rest of the application.

Technical Features

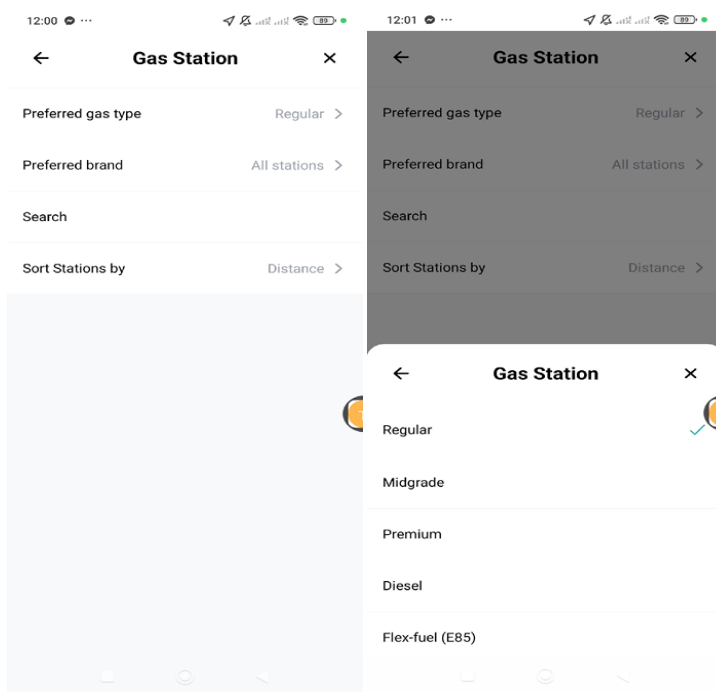
The settings module incorporates multiple navigation and UI techniques to deliver a fluid and responsive experience. It can be accessed through the bottom tab navigator or via the profile modal menu, offering two convenient entry points for users. Each sub-screen within the settings hierarchy is designed to maintain a consistent header structure, including a back button for navigation and a close button for modal dismissal. The team implemented reusable components to maintain uniformity across screens and optimize development efficiency.

State management within this module currently functions locally, preserving user interactions temporarily until backend APIs are connected. The visual design leverages React Native's flexible layout system to ensure responsive performance on different screen sizes and devices. The hierarchical flow of the Settings Menu connects smoothly to submodules like Gas Station and Speedometer screens, which are ready for backend integration once data storage and retrieval mechanisms are finalized.

Gas Station Settings

Gas Station Screen (GasStationScreen.js):

The Gas Station Settings and Preferred Brand Selection screens were implemented by Pravesh Poudel between October 9–11, 2025. These submodules enable drivers to customize their fuel preferences and brand choices. The Gas Station screen allows users to select a preferred fuel type such as Regular, Midgrade, Premium, Diesel, Flex-fuel, or Biodiesel. It also provides options to sort nearby stations by price, distance, or brand. Interactive modals were used to manage these selections, supported by smooth animations for an engaging user experience.



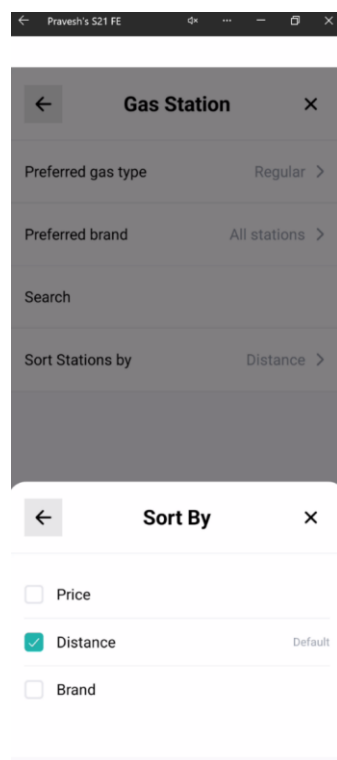
```

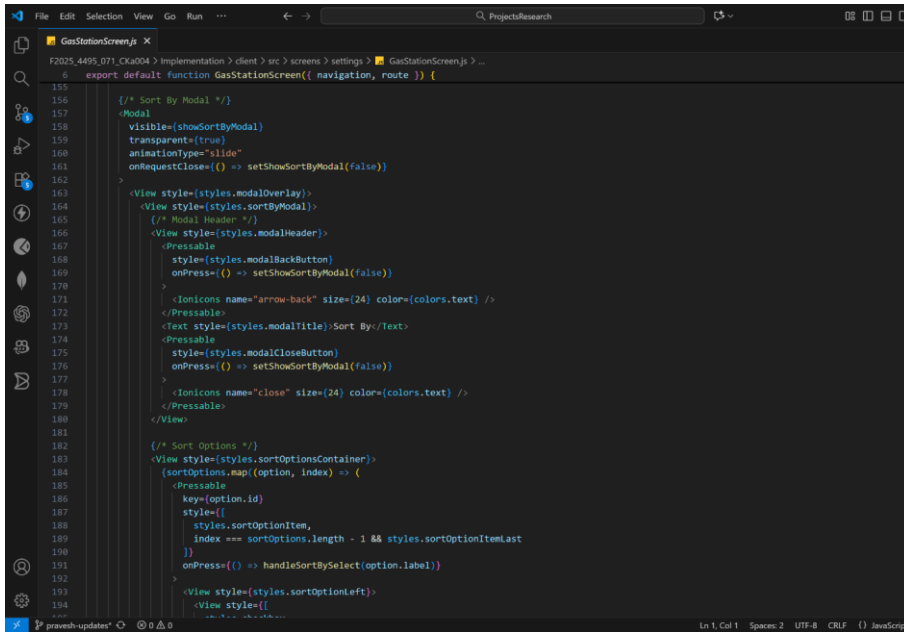
File Edit Selection View Go Run ProjectResearch
GasStationScreen.js X
F2025.4495.071_Cha004 > Implementation > client > src > screens > settings > GasStationScreen.js > ...
1 import React, { useState, useEffect } from "react";
2 import { View, Text, StyleSheet, Pressable, ScrollView, Modal } from "react-native";
3 import { Ionicons } from "@expo/vector-icons";
4 import { colors, spacing, type, radius } from "../../theme/tokens";
5
6 export default function GasStationScreen({ navigation, route }) {
7   const [showGasTypeModal, setShowGasTypeModal] = useState(false);
8   const [showSortByModal, setShowSortByModal] = useState(false);
9   const [selectedGasType, setSelectedGasType] = useState("Regular");
10  const [selectedBrand, setSelectedBrand] = useState("All stations");
11  const [sortBy, setSortBy] = useState("Distance");
12
13  // Update selected brand when returning from PreferredBrandScreen
14  useEffect(() => {
15    if (route.params?.selectedBrand) {
16      setSelectedBrand(route.params.selectedBrand);
17    }
18  }, [route.params?.selectedBrand]);
19
20  const gasTypes = [
21    { id: "regular", label: "Regular", selected: true },
22    { id: "midgrade", label: "Midgrade", selected: false },
23    { id: "premium", label: "Premium", selected: false },
24    { id: "diesel", label: "Diesel", selected: false },
25    { id: "flex-fuel", label: "Flex-fuel (E85)", selected: false },
26    { id: "biodiesel", label: "Biodiesel", selected: false },
27  ];
28
29  const sortOptions = [
30    { id: "price", label: "Price", isDefault: false },
31    { id: "distance", label: "Distance", isDefault: true },
32    { id: "brand", label: "Brand", isDefault: false },
33  ];
34
35  const handleGasTypeSelect = (type) => {
36    setSelectedGasType(type);
37    setShowGasTypeModal(false);
38  };
39
40  const handleSortBySelect = (option) => {
41    setSortBy(option);
  }

```

Sort By Functionality

The Sort By functionality provides users with a flexible and intuitive way to organize displayed results according to their preferences. It includes multiple sorting options such as price-based sorting, allowing users to arrange listings from lowest to highest or vice versa, and distance-based sorting, which helps users prioritize nearby locations for convenience. Additionally, brand-based sorting enables users to filter or organize results according to specific service providers or station brands. Each sorting option is accompanied by visual checkboxes or toggle buttons that make selection quick and clear, providing immediate visual feedback upon user interaction. A default indicator display highlights the currently active sorting method, ensuring users always know which criterion is being applied. This feature enhances usability by promoting clarity, interactivity, and efficient navigation when exploring multiple options within the app.

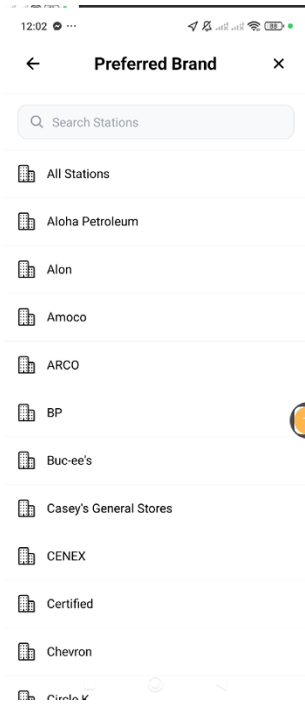




```
export default function GasStationScreen({ navigation, route }) {  
  // Sort By Modal  
  <Modal  
    visible={showSortByModal}  
    transparent={true}  
    animationType="slide"  
    onRequestClose={() => setShowSortByModal(false)}  
  >  
    <View style={styles.modalOverlay}>  
      <View style={styles.sortByModal}>  
        <View style={styles.modalHeader}>  
          <Pressable  
            style={styles.modalBackButton}  
            onPress={() => setShowSortByModal(false)}  
          >  
            <Image style={styles.modalBackImage} source={require('./assets/back.png')} />  
          </Pressable>  
          <Text style={styles.modalTitle}>Sort By</Text>  
          <Pressable  
            style={styles.modalCloseButton}  
            onPress={() => setShowSortByModal(false)}  
          >  
            <Image style={styles.modalCloseImage} source={require('./assets/close.png')} />  
          </Pressable>  
        </View>  
        <View style={styles.sortOptionsContainer}>  
          {sortOptions.map((option, index) => {  
            <Pressable  
              key={option.id}  
              style={styles.sortOptionItem}  
              index={index} <= sortOptions.length - 1 && styles.sortOptionItemLast  
            >  
              <Text style={styles.sortOptionText}>{option.label}</Text>  
              <Pressable  
                style={styles.sortOptionLeft}</Pressable>  
            </Pressable>  
          })}</View>  
      </View>  
    </Modal>  
  </View>  
  <View style={styles.sortOptionsContainer}>  
    {sortOptions.map((option, index) => {  
      <Pressable  
        key={option.id}  
        style={styles.sortOptionItem}  
        index={index} <= sortOptions.length - 1 && styles.sortOptionItemLast  
      >  
        <Text style={styles.sortOptionText}>{option.label}</Text>  
        <Pressable  
          style={styles.sortOptionLeft}</Pressable>  
      </Pressable>  
    })}</View>  
  </View>  
}
```

Preferred Brand Selection Screen (PreferredBrandScreen.js):

The Preferred Brand Selection Screen expands this functionality by allowing users to choose from a searchable list of over 70 gas station chains, including major brands like Shell, Exxon, BP, Chevron, and Sunoco. The interface supports real-time filtering as users type and maintains the selected brand in local state until backend linkage is complete. Upon returning to the main Gas Station screen, the selected brand remains persistent, demonstrating effective use of route parameters and React hooks for temporary data management.



```
1 import React, { useState } from "react";
2 import { View, Text, StyleSheet, Pressable, ScrollView, TextInput } from "react-native";
3 import { Ionicons } from "@expo/vector-icons";
4 import { colors, spacing, type, radius } from "../theme/tokens";
5
6 export default function PreferredBrandScreen({ navigation }) {
7   const [searchText, setSearchText] = useState("");
8
9   const gasBrands = [
10     "All Stations",
11     "Aloha Petroleum",
12     "Alon",
13     "Amoco",
14     "ARCO",
15     "BP",
16     "Buc-ee's",
17     "Casey's General Stores",
18     "CENEX",
19     "Certified",
20     "Chevron",
21     "Circle K",
22     "Citgo",
23     "Clark Brands",
24     "Conoco",
25     "Costco (Kirkland Brand gasoline)",
26     "Crown",
27     "Cumberland Farms",
28     "DK",
29     "EdiWorld",
30     "Exxon",
31     "Family Express - Indiana",
32     "Flying J",
33     "GoAmerica",
34     "Gas City, Ltd.",
35     "Getty",
36     "Go-Mart",
37     "Gulf",
38     "Hale",
39     "High's Dairy Stores",
40     "Holiday",
41     "Irving Oil",
42   ];
```

Code Quality and Organization

Throughout development, the team followed best practices in React Native development to ensure clean, maintainable, and scalable code. Hooks, such as using State and Navigation, were used for efficient state and route management. All screens were structured modularly, with well-commented code blocks, clear variable naming conventions, and meaningful function organization. The component tree was designed to separate logic and UI for easier debugging and future scalability.

Version control was managed through GitHub, where all commits related to this feature were pushed to the `pravesh-updates` branch. Each commit message clearly described the implemented functionality, making the development history traceable and easy to review. Regular updates ensure synchronization between local development and the shared repository, supporting collaboration and code reviews.

Testing and Validation

Testing for the Settings Management System was performed using manual validation on the Expo Go environment. We verified navigation between screens, checked animation behavior, and ensured smooth interaction with modals and menus. Functional checks included verifying state synchronization between the Settings Menu and submodules, confirming search functionality in the brand selection screen, and validating proper persistence of temporary selections. Navigation and back-button behaviors were tested extensively to ensure a consistent and intuitive flow.

Overall, the Settings Management System achieved its goal of providing users with a responsive, well-organized, and customizable interface. While backend data handling is yet to be integrated, the front-end implementation is complete, stable, and ready for further expansion. This module stands as a key element of the Weather Driver app's user experience, enabling personalization and control that align with modern mobile application standards.

Implemented Feature 4: Speedometer and Gas Station Preferences

Speedometer Screen

(SpeedometerScreen.js):

The Speedometer Screen represents one of the most visually dynamic and functional components of the Weather Driver application. It was developed by Chetan Kaur between October 8 -10, 2025, focusing primarily on front-end design and interactive visualization. The feature was implemented to align precisely with the Figma specifications provided by the Client, ensuring that the interface is both aesthetically appealing and practical for real-world driving scenarios.

Features Implemented:

The Speedometer interface displays a responsive circular dial with tick marks, speed readings, and smooth animations. It allows users to toggle between kilometers per hour (km/h) and miles per hour (mph), accommodating international driving standards. The visual design incorporates a modern gradient color scheme and clear typography to maintain readability at a glance. The dial layout, animation behavior, and transition effects were created using React Native's Animated API, providing smooth real-time motion and interaction feedback.

Technical Details:

From a technical perspective, the Speedometer was built using a custom gauge visualization that updates dynamically based on mock speed data. Although real-time GPS speed tracking is not yet integrated, the component was structured with hooks and states that will support live data in future iterations. The layout adapts seamlessly to different screen sizes, maintaining proportional scaling and responsiveness across devices.

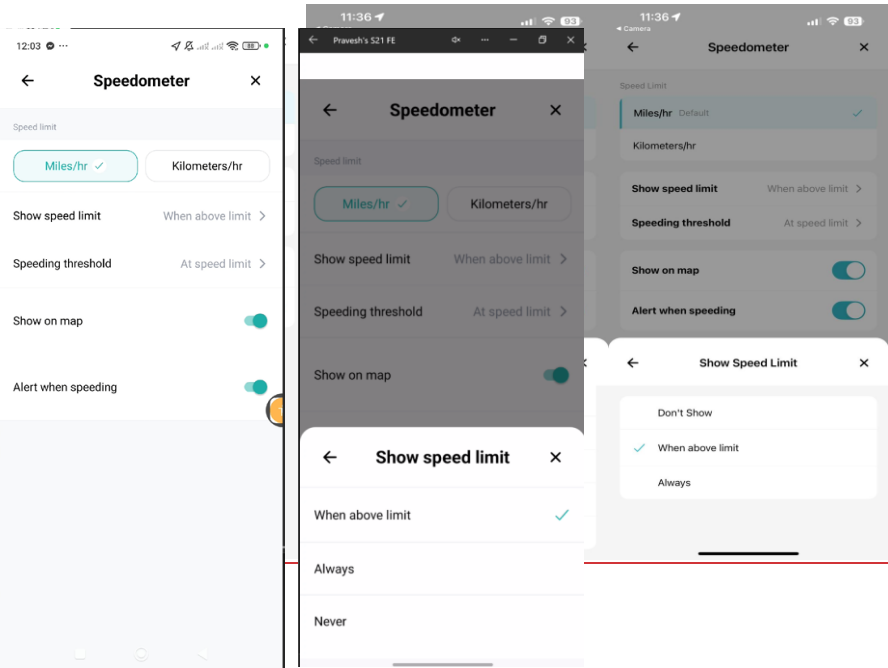
Development Process:

The development process began on October 8, with the initial UI structure and dial elements being created according to the Figma mockup. The next day, work continued the speed unit

toggle, adding functionality for users to switch between km/h and mph. The same day, the first version of the screen was pushed to GitHub under the chetan_300391004 branch, ensuring proper version control and documentation. On October 10, final refinements were made to the layout and animations, including improved dial precision and smoother transitions. The testing phase involved collaboration with Pukar Ojha, during which both developers verified visual consistency, corrected minor UI spacing issues, and validated all navigation links within the app.

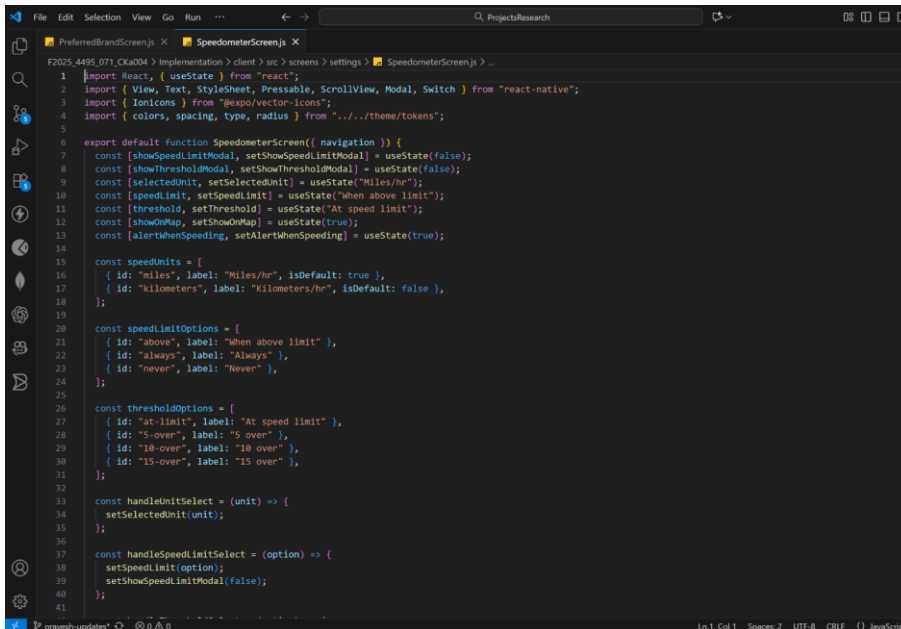
Overall, the Speedometer feature provides the Weather Driver application with an engaging, user-friendly visual tool that will later evolve into a real-time driver assistance component once backend GPS integration is complete.

Screenshots



Formatted: Centered

—



```
1 import React, { useState } from "react";
2 import { View, Text, StyleSheet, Pressable, ScrollView, Modal, Switch } from "react-native";
3 import { Ionicons } from "@expo/vector-icons";
4 import { colors, spacing, type, radius } from "../../theme/tokens";
5
6 export default function SpeedometerScreen({ navigation }) {
7   const [showSpeedLimitModal, setShowSpeedLimitModal] = useState(false);
8   const [showThresholdModal, setShowThresholdModal] = useState(false);
9   const [selectedUnit, setSelectedUnit] = useState("Miles/hr");
10  const [speedLimit, setSpeedLimit] = useState("When above limit");
11  const [threshold, setThreshold] = useState("At speed limit");
12  const [showOnMap, setShowOnMap] = useState(true);
13  const [alertWhenSpeeding, setAlertWhenSpeeding] = useState(true);
14
15  const speedUnits = [
16    { id: "miles", label: "Miles/hr", isDefault: true },
17    { id: "kilometers", label: "Kilometers/hr", isDefault: false },
18  ];
19
20  const speedLimitOptions = [
21    { id: "above", label: "When above limit" },
22    { id: "always", label: "Always" },
23    { id: "never", label: "Never" },
24  ];
25
26  const thresholdOptions = [
27    { id: "at-limit", label: "At speed limit" },
28    { id: "5-over", label: "5 over" },
29    { id: "10-over", label: "10 over" },
30    { id: "15-over", label: "15 over" },
31  ];
32
33  const handleUnitSelect = (unit) => {
34    setSelectedUnit(unit);
35  };
36
37  const handleSpeedLimitSelect = (option) => {
38    setSpeedLimit(option);
39    setShowSpeedLimitModal(false);
40  };
41}
```

Splash Screen Refinement

Alongside the Speedometer development, a major improvement was made to the Splash Screen, which serves as the user's first interaction with the application. The refinement was collaboratively completed by Chetan Kaur and Pukar Ojha on October 10, 2025, following feedback from internal testing and the project sponsor.

The improvements included adjustments to logo scaling to ensure visual balance on all device resolutions and updates to the timing sequence to achieve smoother transitions during app startup. The splash animation duration was optimized to prevent delays during cold starts while maintaining a polished appearance that aligns with brand guidelines. Visual assets, including the Weather Driver logo and background gradients, were updated for improved clarity and consistency with the onboarding flow.

Additional testing was conducted to ensure that the splash screen handled both cold start and resume behaviors correctly. These refinements enhance the app's initial impression and overall professional presentation, contributing to a cohesive brand identity. The collaborative effort

between Chetan and Pukar not only improved the visual flow but also strengthened the reliability of the app's initialization process, which is essential for future deployment on mobile platforms.

Lessons Learned and Future Work

Lessons Learned

Working on the Weather Driver project has been an enriching experience that has strengthened both our technical and collaborative skills. This phase of the project allowed the team to apply theoretical concepts learned throughout the Computer Studies and Information Systems program at Douglas College to a real-world development environment. The process of transforming a client's vision into a functional mobile prototype deepened our understanding of how software design, project management, and user-centered thinking intersect in applied research.

The project's use of Agile with Scrum methodology provided essential insights into iterative development and team coordination. Through sprint planning, daily stand-ups, and retrospectives, we learned to divide complex tasks into manageable deliverables, improve communication, and maintain adaptability in the face of challenges. Encountering issues such as Expo configuration errors, TypeScript-to-JavaScript conversion, and repository synchronization helped us develop resilience and strong troubleshooting skills. Each member gained a clearer perspective on how version control, modular development, and cross-platform testing are managed in a professional workflow.

This project also strengthened our proficiency with React Native, Figma-based UI implementation, and GitHub collaboration, enabling us to build functional mobile interfaces and maintain consistent design patterns. Moreover, working with a real client through Riipen provided valuable professional exposure from conducting requirement analysis meetings to receiving feedback on sprint outcomes. The experience underscored the importance of aligning deliverables with client expectations and reinforced how crucial communication and documentation are in ensuring project success.

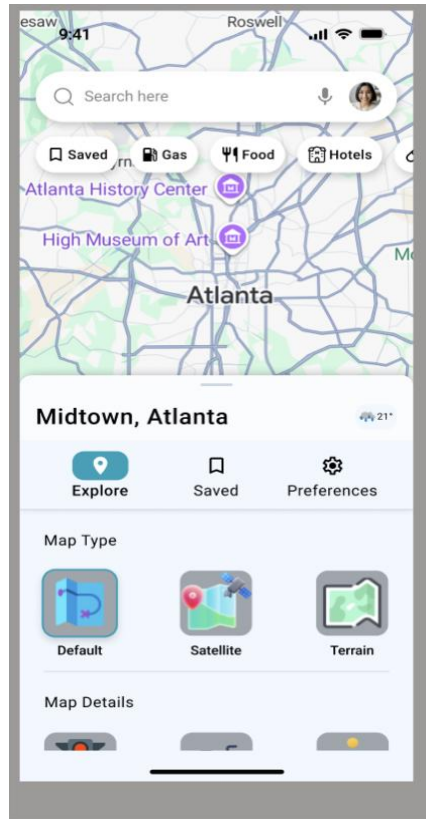
Academically, this project is connected deeply with coursework in software engineering, cloud computing, and applied research. It demonstrated how classroom concepts like modular programming, data abstraction, and agile workflow transition seamlessly into real-world applications. On a personal level, each team member improved their time management, adaptability, and teamwork skills that are vital in any professional technology environment.

Future Work

Moving forward, the next phase of development will focus on backend integration, feature expansion, and UI refinement based on the client's updated requirements. The primary objective will be to connect the existing frontend with AWS services, enabling real-time data synchronization, secure user authentication, and cloud-hosted functionality. Integration with the Google Maps API and NOAA Weather API will allow the app to display live weather alerts, hazard overlays, and real-time environmental updates directly within the map interface, fulfilling one of the core research goals.

In addition to backend connectivity, the team will continue to enhance existing modules to deliver a more complete and dynamic application experience. Planned updates include implementing real-time GPS tracking in the Speedometer, connecting Gasometer readings to live data, and activating the alert notification system to display environmental and travel warnings. The notification screens will be fully developed to manage user alerts efficiently, while the Edit Profile module will be refined to support user data updates once backend APIs are operational. The toll pass feature will also be introduced to provide route-based payment assistance.

The client has recently updated the map interface design, requiring adjustments to the user experience and visual layout. The team will incorporate these new interface changes such as improved search bar placement, map mode options (default, satellite, terrain), and bottom navigation refinements into the next development sprint to align the app with the revised Figma design (as shown in the image reference).



A dedicated testing and optimization phase will follow, focusing on end-to-end functionality, usability testing, and performance tuning to ensure the application is stable, responsive, and ready for deployment. Documentation for both technical and user perspectives will be prepared, alongside a final deployment plan for Android and iOS platforms.

Professionally, this project continues to guide each team member toward specialized expertise in mobile app development, cloud computing, and cybersecurity. The applied research experience has enhanced our teamwork, technical adaptability, and communication skills qualities that will serve us in both future coursework and professional roles. The Weather Driver app will ultimately evolve into a fully cloud-integrated, data-driven, and user-centered mobile solution that bridges academic learning with real-world industry standards.

Concluding Remarks

The *Weather Driver* project has demonstrated substantial progress and collaboration throughout its development journey, transforming an initial concept into a well-structured and visually functional mobile application. By leveraging modern tools such as React Native, Expo, and Figma, the team has successfully implemented key frontend features, including user authentication, core navigation, settings management, and visual interface modules such as the ‘speedometer’ and ‘gas station’ preferences. These accomplishments not only represent strong technical proficiency but also reflect the team’s ability to adapt to challenges, communicate effectively, and maintain alignment with client expectations.

This project has provided each team member with valuable real-world experience in Agile software development, enhancing both technical and interpersonal skills essential for future professional environments. The process of converting research objectives into practical deliverables has strengthened the team’s understanding of mobile system design, API integration planning, and UI/UX best practices. The exposure to client collaboration through Riipen and the opportunity to align academic knowledge with applied research have enriched our learning experience far beyond the classroom.

As the team moves forward, the focus will shift toward backend integration, live data connections, and comprehensive system testing to finalize the Weather Driver application as a complete, cloud-enabled product. This milestone would not have been possible without the guidance of Professor Bambang Sarif, whose support and feedback throughout the course have been invaluable. We would also like to express our sincere gratitude to Douglas College for providing the resources and academic environment necessary to conduct this research, and to our project sponsor, Juliet from Alternative Industries, for her mentorship, feedback, and updated design inputs.

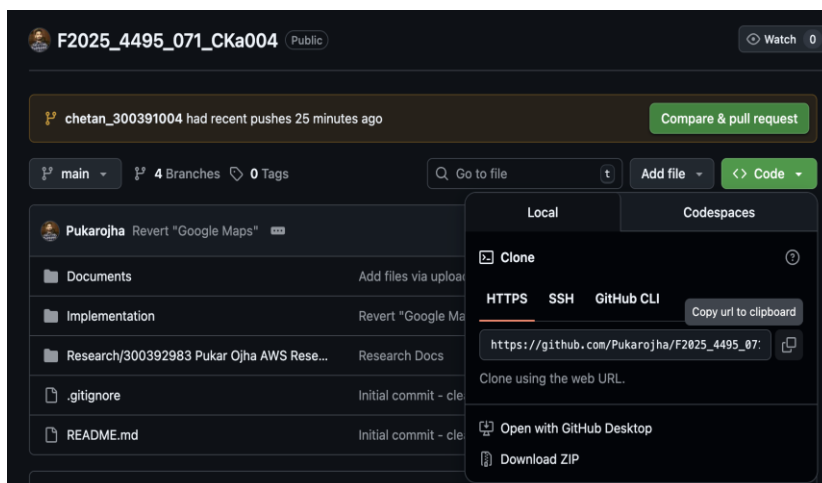
Appendix

Appendix A: Installation Guide

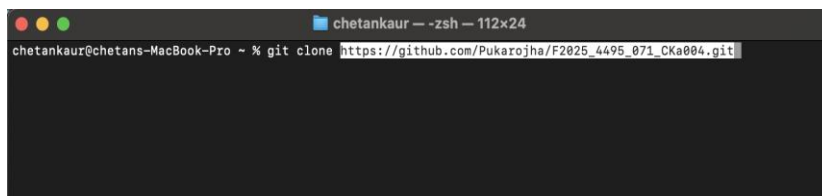
The Weather Driver mobile application is developed using React Native with Expo (managed workflow). It provides users with real-time weather updates, speed monitoring, map visualization, and nearby gas station locations. Follow the steps below to install and run the project.

Step 1: Clone the Repository

- Copy the git repository from the git hub. First, open the repository and click on the clone button and then in the HTTPS tab and then copy the git repository URL.



- In the terminal git clone `https://github.com/Pukarajha/F2025_4495_071_CKa004.git`
`cd F2025_4495_071_CKa004`



- `cd Implementation`

- cd client

```

chetakaur@chetas-MacBook-Pro F2025_4495_071_Cka004 % ls
Documents      Implementation  README.md      Research
chetakaur@chetas-MacBook-Pro F2025_4495_071_Cka004 % cd Implementation
chetakaur@chetas-MacBook-Pro Implementation % ls
client          FigmaLink.txt  package-lock.json  server
chetakaur@chetas-MacBook-Pro Implementation % cd client
chetakaur@chetas-MacBook-Pro client % npx expo start --tunnel

```

Step 2: Install Dependencies

Install the required dependencies using either npm or yarn:

- npm install
- or
- yarn install

Step 3: Set Up Environment Variables

Create a .env file in the root directory and include the following variables:

```

WEATHER_API_KEY=yet_to_be_provided
MAPS_API_KEY=figd_guHA6aNuQFKBvpkQ1CvVhgVCe2c2cPpkhYfCTpa1

```

These variables can be accessed in the app using expo-constants or react-native-dotenv.

Step 4: Configure App Permissions and API Keys

Since the application is developed using Expo, all permissions and API configurations are managed through the app.config.js file instead of native files such as AndroidManifest.xml or Info.plist.

This configuration ensures that all permissions and API keys are automatically bundled during the Expo build process.

Step 5: Run the Application

Start the Expo development server:

-npx expo start --tunnel

Then, follow these steps:

- Open the Expo Go app on your mobile device.
- Scan the QR code displayed in the terminal or on the Expo Dev Tools webpage.
- The Weather Driver app will launch automatically on your device.

Appendix B: User Guide

The Weather Driver app enables users to view live weather data, track their current speed, view maps, and find nearby gas stations with real-time data integration.

Core Features

- Sign Up: Register a new account using email and password, or through social logins (Facebook, Google, or Apple (yet to be implemented)).
- Login: Authenticate existing users and request location permission for real-time tracking.
- Map: Shows the user's location with an interactive Google Map.
- Speedometer: Tracks and displays speed using GPS data.
- Gas Stations: Displays a list of nearby gas stations using the Google Places API.

Steps to Use

- Launch the app using the Expo Go application.
- Allow location permissions when prompted.
- Navigate to the Map Tab to view your current route and nearby gas stations.

- Use the Alerts Tab to view categorized weather alerts (Extreme, Severe, Moderate, Minor).
- In the Search Tab, explore locations and apply filters for Saved, Gas, Food, or Hostel options.
- Under the Settings Tab, access the following sections:
- Notifications: Configure phone and email alert settings.
- Driving Preferences:
- Gas Station Settings: Filter based on preferred gas type, brand, or sort by distance, price, or brand.
- Speedometer: Displays your real-time speed with options to toggle between miles/hr and km/hr, including alerts for exceeding speed limits.
- Profile Icon: Provides access to username and email information, along with options to Plan a Drive, Inbox, Settings, Notifications, Help & Support, and Logout.

Appendix C: Dataset and APIs Used

API	Description	Source
OpenWeatherMap API	Provides real-time weather data and forecasts.	https://openweathermap.org/api
Google Maps API	Displays interactive maps and user location.	https://developers.google.com/maps
Google Places API	Retrieves nearby gas station data.	https://developers.google.com/maps/documentation/places
Expo Location API	Accesses real-time device GPS data.	https://docs.expo.dev/versions/latest/sdk/location/

Appendix D: Hardware, Software, and System Architecture

Hardware Requirements

- Android (10+) or iOS (14+) smartphone
- GPS and Internet connectivity enabled

Software Requirements

- Node.js (v16 or above)
- Expo CLI
- Expo Go app (for mobile testing)
- Visual Studio Code (for development)
- System Architecture Overview
- Frontend: React Native with Expo
- Backend: No dedicated backend (uses external REST APIs)

Data Flow:

User Request → API Fetch → JSON Response → UI Render

State Management: React Context API / Hooks

Data Sources: Weather and Places APIs via Axios REST calls

Appendix E: Backend Infrastructure and AWS Integration Research

Overview

A significant part of our midterm development period was dedicated to understanding and preparing the AWS backend infrastructure that will support the Weather Driver application. This research was essential for ensuring our frontend implementation aligns with backend capabilities.

Learning Activities:

September 30, 2025: Client meeting where AWS upskilling was requested

October 1-2, 2025: Completed client-shared AWS video training and official AWS documentation study focusing on:

- AWS Amplify (Authentication & Hosting)
- Amazon Cognito (User management)
- AWS Lambda (Serverless business logic)
- DynamoDB (NoSQL database)
- API Gateway (REST API management)
- LocalStack (Local AWS emulation)

October 6, 2025: Additional tutorial video review and supplementary research on:

- Hosting configurations
- Authentication flows
- API integration patterns
- Cloud service architecture

Practical AWS Implementation

October 7, 2025: Created experimental GitHub repository

Repository: <https://github.com/Pukarojha/wildrydes-site.git>

Purpose:

- Safe environment for AWS service testing
- Practice deployments without affecting main project
- Configuration testing
- Service connection validation

October 9, 2025: Breakthrough - successful test deployment achieved

Google Maps API Research

Date: October 11, 2025

Areas Explored:

- API key management and security
- Marker placement and customization
- Dynamic route generation
- Geolocation features
- Distance calculations
- Place search functionality
- Directions API integration

Documentation Created:

- Comprehensive research report: "Deployment of the app.docx"
- Google Maps API integration notes
- Code examples and best practices
- Links to experimental repositories

Backend Architecture

Existing Backend Structure:

The server folder contains a Python-based backend with:

AWS Lambda Functions:

- `'alert_listener'` - Weather alert processing
- `'cleanup'` - Data cleanup operations
- `'poll_alerts'` - Active alert monitoring
- `'search'` - Location search functionality

- `zone\_backfill` - Weather zone data population
- `zone\_listener` - Zone update monitoring

Library Modules:

- `WeatherAlertModel.py` - Data model for weather alerts
- `alert\_service.py` - Alert business logic
- `dynamo\_client.py` - DynamoDB interface
- `http\_client.py` - HTTP request handling
- `noaa\_client.py` - NOAA API integration
- `s3\_client.py` - S3 storage operations
- `sqs\_client.py` - Message queue operations
- `zone\_service.py` - Weather zone management

Infrastructure as Code:

- Terraform configurations for local and production environments
- Resource definitions for Lambda, DynamoDB, S3, SQS
- Automated deployment scripts

Integration Planning

Frontend-Backend Connection Strategy:

1. Authentication Flow:

- Frontend: React Native auth screens
- Backend: Amazon Cognito user pools
- Token management: AWS Amplify SDK integration

2. Weather Data Flow:

- Frontend: Map screen weather overlay
- Backend: Lambda functions → NOAA API → DynamoDB caching
- Real-time updates: WebSocket or polling mechanism

3. User Preferences:

- Frontend: Settings screens
- Backend: DynamoDB user preferences table
- Sync: On settings change and app launch

Technical Achievements

Knowledge Gained:

- Cloud-based serverless architecture

- AWS service integration patterns
- Authentication and authorization flows
- NoSQL database design
- API Gateway configuration
- Infrastructure as Code with Terraform

Appendix F: Challenges and solutions

1. Technology Stack Discrepancies

Challenge: Mismatch between client's requirement document (JavaScript/Node.js) and actual codebase (TypeScript/Python)

Discovery Date: September 18-20, 2025

Impact:

- Team unfamiliarity with TypeScript
- Potential learning curve affecting timeline
- Code conversion requirements

Solution:

- Meeting scheduled with client technical team (Sept 24, 2025)
- Client agreed to JavaScript implementation
- Focus shifted to feature development rather than language learning
- Conversion strategy established

Outcome: Team proceeded with JavaScript implementation, maintaining productivity

2. Development Environment Issues

Challenge: Expo build errors are preventing application launch

Discovery Date: September 20, 2025

Details:

- Landing page loaded successfully
- Login functionality not working
- Navigation errors
- Package version conflicts

Solution Process:

- Analyzed error messages and stack traces
- Researched solutions on Stack Overflow
- Identified package version incompatibilities
- Updated package.json dependencies
- Cleared node_modules and reinstalled
- Rebuilt application

Resolution Date: September 26, 2025

Outcome: Stable development environment established for all team members

3. Figma Design Limitations**Challenges Identified:****a. Non-Interactive Designs:**

- Figma files are not interactive
- Difficult to understand the complete user flow
- Screen connections unclear

b. Incomplete Screens:

- Policy screen is inaccurate
- Navigation bar missing in some designs
- Inconsistent design patterns

c. Design Updates During Development:

- Map screens revised (early October)
- Notification screens changed
- General settings screens updated
- Edit profile screen questions

Impact:

- Implementation of delays for affected screens
- Need to revisit completed work
- Questions requiring client clarification

Solution Approach:

- Regular client meetings for clarification
- Screenshot analysis and documentation
- Team collaboration to interpret designs
- Flexible implementation allowing for changes
- Client feedback integration (Sept 30, Oct 11 meetings)

Current Status: Ongoing communication with client; some screens paused pending design finalization.

4 Backend Understanding and Integration

Challenge: Understanding complex AWS architecture and integration requirements

Initial Situation:

- Team unfamiliar with AWS services
- Complex serverless architecture
- Multiple service interactions
- Infrastructure as Code with Terraform

Learning Approach:

- Client-provided video tutorials
- Official AWS documentation study
- Experimental repository creation
- Research documentation

Progress:

- Comprehensive understanding achieved
- Successful test deployment completed
- Documentation created for team reference
- Integration strategy defined

Ongoing: Full integration with the production environment is scheduled for the next phase after the completion of UI development.

5 GitHub Collaboration and Version Control

Challenge: Code conflicts and package version issues in the shared repository

Incident Date: September 27, 2025

Details:

- Group member commit conflicted with package versions

- Repository disrupted for all team members
- Code synchronization issues

Resolution:

- Package.json and package-lock.json reconciliation
- Branch strategy review
- Conflict resolution using Git tools

Prevention Measures Implemented:

- Individual feature branches for each developer
- chetan_300391004 branch (Chetan)
- pravesh-updates branch (Pravesh)
- pukar-dev branch (Pukar)
- Pull request review process
- Communication before merging into the main
- Regular git status checks

Appendix G: Next Steps and Future Work

1. Additional Features:

- Complete Edit Profile screen (pending Figma clarification from Oct 11 meeting)
- Implement notification screens
- Build toll pass management screens

2. Settings Completion:

- Implement General Settings screen
- Build Map Display Settings
- Create Voice and Sound Settings
- Develop Notification preference screens

3. Backend Integration:

- Connect frontend authentication to AWS Cognito
- Implement token management
- Test login/signup with the actual backend

4. Map Enhancement:

- Integrate Google Maps API with API key
- Implement real-time GPS tracking
- Add location markers
- Test map interactions

5. Weather Alert Integration:

- Connect to NOAA API via backend Lambda functions
- Display alerts on the Alerts screen
- Implement alert badge notifications
- Test alert card rendering

6. Advanced Features:

- Route planning with weather considerations
- Gas station finder with real-time prices
- Turn-by-turn navigation preparation

7. Focused Areas development:

- Weather overlay on the map
- Route optimization based on weather
- Push notification system
- User profile management
- Driving history tracking

- Favorite locations
- Advanced search features

8. Polish and Optimization:

- Animation improvements
- Loading state handling
- Error handling refinement
- Offline mode consideration

9. Testing Phase:

- Integration testing
- UI/UX refinements
- Bug fixes
- Unit testing implementation
- Integration testing
- User acceptance testing preparation
- Performance benchmarking

Final Deliverables:

- Fully functional mobile application
- Backend deployed to production AWS
- Comprehensive test results
- User documentation
- Technical documentation
- Final presentation
- Project demonstration
- Code repository handover

Known Issues to Address

- **Backend Connection:** Frontend screens ready but not yet connected to AWS services
- **Weather Data:** NOAA API integration pending
- **Real-time Updates:** WebSocket or polling mechanism needs implementation
- **Offline Mode:** Data caching strategy required
- **Performance:** Optimization needed for map rendering with weather layers
- **Testing:** Automated testing framework setup pending

Key Achievements

- Completed 8 authentication-related screens with full validation and navigation flow
- Established four-tab navigation system (Map, Alerts, Search, Settings)
- Implemented sophisticated settings management with multiple sub-screens
- Built gas station preferences with searchable brand selection (70+ brands)
- Developed speedometer interface with customization options
- Achieved AWS deployment in experimental environment
- Researched and documented Google Maps API integration
- Maintained regular client communication with weekly meetings
- Established efficient team collaboration with individual branches and code review

Team Collaboration Highlights

The project demonstrates strong team collaboration with complementary skills and responsibilities:

Chetan Kaur: Authentication screens, speedometer implementation, AWS learning, UI refinements

Pukar Ojha: Repository management, AWS research and deployment, splash screen, technical problem-solving

Pravesh Poudel: Navigation architecture, settings system, gas station features, map interface

Appendix H: Code Explanation

1 Key Files and Their Functions

1.1 Root Level Files

App.js (Root Application File)

Location: `Implementation/client/App.js`

Purpose: Application entry point, navigation configuration, global providers

Key Responsibilities:

- Set up React Navigation (Stack & Tab navigators)
- Configure QueryClient for data fetching
- Define navigation structure
- Apply global theme
- Configure StatusBar

Dependencies Used:

```
- `@react-navigation/native`  
- `@react-navigation/native-stack`  
- `@react-navigation/bottom-tabs`  
- `@tanstack/react-query`  
- `@expo/vector-icons`
```

package.json

Location: Implementation/client/package.json`

Purpose: Project metadata and dependency management

Core Dependencies:

```
json  
{  
  "dependencies": {  
    "@react-navigation/bottom-tabs": "^7.4.7",
```

```

"@react-navigation/native": "^7.1.17",
"@react-navigation/native-stack": "^7.3.26",
"@tanstack/react-query": "^5.90.2",
"expo": "~54.0.10",
"expo-font": "~14.0.8",
"expo-linear-gradient": "~15.0.7",
"expo-location": "~19.0.7",
"react": "19.1.0",
"react-native": "0.81.4",
"react-native-maps": "1.20.1",
"react-native-safe-area-context": "~5.6.0"
}
}

```

1.2 Theme System

tokens.js

Location: 'Implementation/client/src/theme/tokens.js'

Purpose: Centralized design system - colors, typography, spacing, shadows

Why Important: Ensures consistent styling across all screens

1.2.1 Design Tokens Structure:

a. Colors:

javascript

```

export const colors = {
  // Primary Brand Colors
  primary: "#20B2AA",      // Teal (main brand color)
  primaryVariant: "#4DDDD7", // Light teal

```

```
// Background Colors
```

```
bg: "#F8F9FA",      // Light gray background
```

```
surface: "FFFFFF",   // White surface
```

```
// Text Colors
```

```
text: "#000000",     // Primary text
```

```
textSecondary: "#6B7280", // Secondary text
```

```
muted: "#9CA3AF",    // Muted/disabled text
```

```
// UI Colors
```

```
border: "#E5E7EB",   // Border color
```

```
// Status Colors
```

```
success: "#10B981",  // Green
```

```
warning: "#F59E0B",   // Orange
```

```
danger: "#EF4444",    // Red
```

```
// Weather-specific Colors
```

```
lightGreen: "#A3D977",
```

```
lightBlue: "#87CEEB",
```

```
yellow: "#F4D03F",
```

```
purple: "#B19CD9"
```

```
};
```

b. Typography:

```
javascript
```

```
export const type = {  
  h1: { fontSize: 28, lineHeight: 34, fontWeight: "700", fontFamily: "Roboto" },  
  h2: { fontSize: 22, lineHeight: 28, fontWeight: "700", fontFamily: "Roboto" },  
  h3: { fontSize: 18, lineHeight: 24, fontWeight: "600", fontFamily: "Roboto" },  
  body: { fontSize: 16, lineHeight: 22, fontWeight: "400", fontFamily: "Roboto" },  
  caption: { fontSize: 12, lineHeight: 16, fontWeight: "400", fontFamily: "Roboto" },  
  buttonLabel: { fontSize: 16, lineHeight: 20, fontWeight: "600", fontFamily: "Roboto" }  
};
```

c. Spacing:

javascript

```
export const spacing = {  
  xs: 6,  
  sm: 10,  
  md: 14,  
  lg: 18,  
  xl: 24,  
  xxl: 32  
};
```

d. Border Radius:

javascript

```
export const radius = {  
  xs: 8,  
  sm: 12,  
  md: 16,  
  lg: 20,  
  xl: 28,
```

```
pill: 999
};
```

e. Shadows:

```
javascript
```

```
export const shadows = {
  main: {
    shadowColor: "#000",
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
    shadowRadius: 8,
    elevation: 5
  },
  floatingAction: {
    shadowColor: "#000",
    shadowOffset: { width: 0, height: 4 },
    shadowOpacity: 0.2,
    shadowRadius: 12,
    elevation: 15
  }
};
```

Usage Example:

```
```javascript
import { colors, type, spacing, radius } from "../../theme/tokens";

const styles = StyleSheet.create({
```

```
container: {
 backgroundColor: colors.bg,
 padding: spacing.lg
},
title: {
 ...type.h2,
 color: colors.text
},
button: {
 backgroundColor: colors.primary,
 borderRadius: radius.md,
 padding: spacing.md
}
});
```

## 2. Component Library Documentation

### 2.1 WDIInput Component

**Location:** Implementation/client/src/components/ui/WDInput.js`

**Purpose:** Reusable, styled text input with validation and password toggle

**Features:**

- Optional label
- Placeholder text
- Secure text entry for passwords
- Password visibility toggle (eye icon)
- Validation checkmark indicator
- Focus state styling
- Customizable keyboard type
- Auto-capitalization control

### Props:

```
javascript
{
 label: string, // Optional label text
 placeholder: string, // Placeholder text
 value: string, // Current input value
 onChangeText: function, // Value change handler
 secureTextEntry: boolean, // Hide text (for passwords)
 showPasswordToggle: boolean, // Show eye icon
 keyboardType: string, // 'default', 'email-address', etc.
 autoCapitalize: string, // 'none', 'sentences', etc.
 showValidation: boolean, // Show validation checkmark
 isValid: boolean, // Validation state
 style: object // Additional styles
}
```

### Visual States:

1. **Default:** White background, gray border
2. **Focused:** Primary color border (teal), thicker border
3. **Valid:** Green border with checkmark icon
4. **Error:** Red border (when invalid)

### Code Example:

```
javascript
<WDInput
 label="Email"
 placeholder="Enter your email"
```

```
value={email}

onChangeText={setEmail}

keyboardType="email-address"

autoCapitalize="none"

showValidation={true}

isValid={emailValid}

/>
```

```
<WDInput

label="Password"

placeholder="Enter password"

value={password}

onChangeText={setPassword}

secureTextEntry={true}

showPasswordToggle={true}

/>
```

#### **Styling Details:**

- Height: 48px (minimum)
- Border radius: 16px (from tokens.radius.md)
- Padding: 14px horizontal (from tokens.spacing.md)
- Font: Roboto 16px (from tokens.type.body)

## **2.2 WDButton Component**

**Location:** `Implementation/client/src/components/ui/WDButton.js`

**Purpose:** Reusable, styled button component with variants

**Features:**

- Primary variant (filled)
- Secondary variant (outlined)
- Press state feedback
- Customizable styling
- Consistent with design tokens

#### Props:

```
javascript
{
 label: string, // Button text
 onPress: function, // Press handler
 variant: 'primary' | 'secondary',
 style: object, // Additional button styles
 textStyle: object // Additional text styles
}
```

#### Variants:

##### Primary Button:

- Background: Teal (#20B2AA)
- Text: White
- Height: 48px
- Border radius: 16px
- Use case: Main actions (Sign In, Submit, Save)

##### Secondary Button:

- Background: Transparent
- Border: 1px gray
- Text: Black
- Use case: Cancel, alternative actions

#### Code Example:

```
javascript
<WDButton
```

```
label="Sign In"

onPress={handleSignIn}

variant="primary"

/>
```

```
<WDButton

label="Cancel"

onPress={handleCancel}

variant="secondary"

/>
```

**Press Feedback:**

- Opacity: 0.8 when pressed
- Provides tactile feedback to user

## 2.3 RouteModal Component

**Location:** `Implementation/client/src/components/RouteModal.js`

**Purpose:** Modal for route planning and navigation

**Used In:** MapScreen.js

**Features:**

- Slide-up animation
- Origin/destination display
- Route details
- Close button
- Backdrop overlay

## 2.4 AlertBadge Component

**Location:** `Implementation/client/src/components/AlertBadge.js`

**Purpose:** Display alert severity badges

**Used In:** AlertScreen.js

**Severity Levels:**

- Extreme (Red)
- Severe (Orange)
- Moderate (Yellow)
- Minor (Green)

### 3. Screen Implementation Details

#### 3.1 Authentication Screens

**SplashScreen.js**

**Purpose:** App loading screen with branding

**Duration:** 2-3 seconds

**Content:**

- Weather Driver logo
- Cloud imagery
- Loading indicator

**Navigation Flow:**

SplashScreen → OnboardingScreen (first launch)

→ SignInScreen (returning user)

**SignInScreen.js**

**Purpose:** User login

**Form Fields:**

- Email (WDInput with email keyboard)
- Password (WDInput with secure entry and toggle)

**Validation:**

- Email format check
- Password minimum length
- Required field validation
- Real-time error messages

**Actions:**

- Sign In button → Navigate to MainTabs
- Forgot Password link → ForgotPasswordScreen
- Sign Up link → SignupScreen
- "Remember me" checkbox

**Code Pattern:**

javascript

```
const [email, setEmail] = useState("");

const [password, setPassword] = useState("");

const [emailError, setEmailError] = useState("");

const [passwordError, setPasswordError] = useState("");

const validateEmail = (email) => {

 const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

 return re.test(email);

};
```

```
const handleSignIn = () => {
 if (!validateEmail(email)) {
 setEmailError("Invalid email format");
 return;
 }
 if (password.length < 8) {
 setPasswordError("Password must be at least 8 characters");
 return;
 }
 // Proceed with authentication
 navigation.replace("MainTabs");
};
```

### SignupScreen.js

**Purpose:** New user registration

#### Form Fields:

- Full Name
- Email
- Password
- Confirm Password
- Terms & Conditions checkbox

#### Validation Rules:

- Email: Valid format, unique
- Password: Minimum 8 characters, contains number, special character
- Confirm Password: Must match password
- All fields required

#### Validation States:

- Empty: No styling
- Invalid: Red border, error message
- Valid: Green border, checkmark icon

### **ForgotPasswordScreen.js**

**Purpose:** Initiate password reset

**Flow:**

1. Enter email address
2. Submit → Send verification code
3. Navigate to ForgotPasswordCodeScreen

### **ForgotPasswordCodeScreen.js**

**Purpose:** Enter verification code

**Features:**

- 6-digit code input
- Resend code button (with timer)
- Code validation

### **ResetPasswordScreen.js**

**Purpose:** Set new password

**Features:**

- New password input
- Confirm password input
- Password strength indicator
- Same validation as sign-up

### **PasswordChangedScreen.js**

**Purpose:** Confirmation of successful password reset

**Action:** Navigate to SignInScreen

## **5.2 Main Application Screens**

## **MapScreen.js**

**Purpose:** Main map interface with weather overlay

**Location:** `Implementation/client/src/screens/main/MapScreen.js`

### **Key Features:**

#### **1. Interactive Map:**

- Google Maps integration (react-native-maps)
- User location tracking (expo-location)
- Current location marker
- Weather zone polygons
- Alert markers
- Custom map styling

#### **2. Search Bar (Top):**

- Search icon
- Text input placeholder
- Voice search icon
- Profile avatar button

#### **3. Profile Avatar & Modal:**

**Location in Code:** Lines 95-236

#### **Avatar Display:**

- Circular avatar with initials
- Located in search bar (top right)
- Color: Primary teal

#### **Profile Menu Options:**

1. **Plan a drive** → Opens RouteModal
2. **Inbox** → Message center

3. **Settings** → SettingsMenuScreen
4. **Notification** → Notification preferences
5. **Help and Support** → Help center
6. **Logout** → Returns to SignInScreen

#### Menu Implementation:

```
const [showProfileMenu, setShowProfileMenu] = useState(false);
```

```
const handleProfileMenuPress = (option) => {
 setShowProfileMenu(false);
 if (option === "Logout") {
 navigation.replace("SignIn");
 } else if (option === "Plan a drive") {
 setShowRouteModal(true);
 } else if (option === "Settings") {
 navigation.navigate("Settings");
 }
};
```

#### 4. Floating Action Buttons:

- **Route Button** (bottom right): Opens route planning modal
- **Location Button** (below route): Centers map on user location

#### 5. GPS Location Tracking:

```
useEffect(() => {
 (async () => {
 const { status } = await Location.requestForegroundPermissionsAsync();
 if (status !== 'granted') return;
 const loc = await Location.getCurrentPositionAsync({});
 setRegion({
 latitude: loc.coords.latitude,
```

```
 longitude: loc.coords.longitude,
 latitudeDelta: 0.25,
 longitudeDelta: 0.25
 });
 }());
}, []);
```

## 6. Route Modal:

- Slide-up modal
- Origin/destination input
- Route options
- Weather warnings along route

### AlertScreen.js

**Purpose:** Display list of weather alerts **Components Used:**

- AlertBadge.js
- AlertCard.js

### Alert Information:

- Severity level
- Alert type
- Location
- Time issued
- Description
- Affected area

### Future Integration:

- Real-time alerts from NOAA API
- Push notifications
- Alert filtering by severity

## **SearchScreen.js**

**Purpose:** Search for locations and addresses

### **Features:**

- Search input
- Recent searches
- Search results list
- Location selection → Update map

## **SettingsMenuScreen.js**

**Purpose:** Main settings navigation hub

**Location:** Implementation/client/src/screens/main/SettingsMenuScreen.js

### **Settings Organization:**

#### **Section 1: Settings**

- General Settings (future)
- Map Display Settings (future)
- Voice and Sound Settings (future)

#### **Section 2: Notifications**

- Phone Notifications (future)
- Email Notifications (future)

#### **Section 3: Driving Preferences**

- Gas Station Settings → GasStationScreen
- Speedometer Settings → SpeedometerScreen

### **UI Pattern:**

- Section headers (gray text)
- Menu items with icons
- Chevron indicators for navigation
- Divider lines between items

**Navigation Integration:**

- Accessible from bottom tab bar
- Accessible from profile modal
- Clean back navigation

**Settings Sub-screens****GasStationScreen.js**

**Purpose:** Gas station preferences and filtering

**Location:** Implementation/client/src/screens/settings/GasStationScreen.js

**Features Implemented:**

**1. Preferred Gas Type Selection Trigger:** Tap "Preferred gas type" row

**Modal Opens:** Gas type selection modal

**Gas Type Options:**

- Regular
- Midgrade
- Premium
- Diesel
- Flex-fuel (E85)
- Biodiesel

**Selection Pattern:**

- List of options in modal
- Selected option shows checkmark
- Tap to select
- Modal closes automatically
- Main screen updates with selection

#### Code Implementation:

```
const [selectedGasType, setSelectedGasType] = useState("Regular");
const [showGasTypeModal, setShowGasTypeModal] = useState(false);
const gasTypes = [
 { id: "regular", label: "Regular", selected: true },
 { id: "midgrade", label: "Midgrade", selected: false },
 { id: "premium", label: "Premium", selected: false },
 { id: "diesel", label: "Diesel", selected: false },
 { id: "flex-fuel", label: "Flex-fuel (E85)", selected: false },
 { id: "biodiesel", label: "Biodiesel", selected: false },
];
const handleGasTypeSelect = (type) => {
 setSelectedGasType(type);
 setShowGasTypeModal(false);
};
```

#### 2. Preferred Brand Selection Trigger: Tap "Preferred brand" row

**Action:** Navigate to PreferredBrandScreen

**Data Flow:** Selected brand passed back via route params

#### 3. Sort Stations by Trigger: Tap "Sort Stations by" row

**Modal Opens:** Sort options modal

#### Sort Options:

- **Price** - Sort by lowest price
- **Distance** - Sort by nearest (DEFAULT)

- **Brand** - Sort alphabetically by brand

#### Modal Features:

- Checkbox for each option
- "Default" label on Distance option
- Visual checkmark when selected
- Tapping option selects and closes modal

#### Code Implementation:

```
const [sortBy, setSortBy] = useState("Distance");
const [showSortByModal, setShowSortByModal] = useState(false);
const sortOptions = [
 { id: "price", label: "Price", isDefault: false },
 { id: "distance", label: "Distance", isDefault: true },
 { id: "brand", label: "Brand", isDefault: false },
];

const handleSortBySelect = (option) => {
 setSortBy(option);
 setShowSortByModal(false);
};

// In render:
{sortOptions.map((option) => (
 <Pressable onPress={() => handleSortBySelect(option.label)}>
 <View style={styles.checkbox}>
 {sortBy === option.label && (
 <Icons name="checkmark" size={16} color="#fff" />
)}
 </View>
 <Text>{option.label}</Text>
 {option.isDefault && <Text style={styles.defaultLabel}>Default</Text>}
 </Pressable>
))}
```

**Modal Styling:**

- Slide-up animation from bottom
- Semi-transparent backdrop
- Rounded top corners
- Header with back/close buttons
- Content area with scrollable options

**4. Search Functionality Status:** Placeholder (future implementation)

**Purpose:** Search for specific gas stations by name/location

## **PreferredBrandScreen.js**

**Purpose:** Select preferred gas station brand

**Location:** Implementation/client/src/screens/settings/PreferredBrandScreen.js

### **Key Features:**

#### **1. Search Functionality**

- Text input at top
- Real-time filtering
- Case-insensitive search
- Instant results

#### **2. Brand Database (70+ Brands)** Major brands included:

- Shell
- Exxon
- BP
- Chevron
- Mobil
- Citgo
- Sunoco
- Texaco
- Valero
- Marathon
- Phillips 66
- Conoco
- 76
- Arco
- Speedway
- And 55+ more regional brands

#### **3. Selection Mechanism**

- Tap brand to select
- Returns to GasStationScreen
- Selected brand displayed on main screen
- Data passed via navigation params

#### Code Implementation:

```
const [searchQuery, setSearchQuery] = useState("");
const [selectedBrand, setSelectedBrand] = useState("All stations");
const brands = [
 "All stations",
 "Shell", "Exxon", "BP", "Chevron", "Mobil",
 "Citgo", "Sunoco", "Texaco", "Valero", "Marathon",
 // ... 60+ more brands
];
const filteredBrands = brands.filter(brand =>
 brand.toLowerCase().includes(searchQuery.toLowerCase())
);
const handleBrandSelect = (brand) => {
 setSelectedBrand(brand);
 navigation.navigate("GasStation", { selectedBrand: brand });
};
```

#### 4. State Persistence Uses React Navigation route params:

```
// In GasStationScreen.js
useEffect(() => {
 if (route.params?.selectedBrand) {
 setSelectedBrand(route.params.selectedBrand);
 }
}, [route.params?.selectedBrand]);
```

#### UI Design:

- Search bar at top

- Scrollable brand list
- Checkmark for selected brand
- Clean, minimal design
- Fast, responsive filtering

## SpeedometerScreen.js

**Purpose:** Configure speedometer display and alerts

**Location:** Implementation/client/src/screens/settings/SpeedometerScreen.js

### Features Implemented:

#### 1. Speed Unit Selection Options:

- Miles/hr (Default)
- Kilometers/hr

### UI Pattern:

- Toggle buttons (pill-style)
- Active state: Primary color background
- Inactive state: White background with border
- Checkmark icon on default/active option

### Code:

```
const [selectedUnit, setSelectedUnit] = useState("Miles/hr");
const speedUnits = [
 { id: "miles", label: "Miles/hr", isDefault: true },
 { id: "kilometers", label: "Kilometers/hr", isDefault: false },
];
<View style={styles.unitSelectorContainer}>
```

```

{speedUnits.map((unit) => (
 <Pressable
 key={unit.id}
 style={[
 styles.unitButton,
 selectedUnit === unit.label && styles.unitButtonActive
]}
 onPress={() => setSelectedUnit(unit.label)}
 >
 <Text style={[
 styles.unitButtonText,
 selectedUnit === unit.label && styles.unitButtonTextActive
]}>
 {unit.label}
 </Text>
 {unit.isDefault && selectedUnit === unit.label && (
 <View style={styles.defaultBadge}>
 <Icons name="checkmark" size={16} color={colors.primary} />
 </View>
)}
 </Pressable>
)}}
</View>

```

## 2. Show Speed Limit Options:

- When above limit
- Always
- Never

### Modal Implementation:

- Tap to open modal
- List of options

- Checkmark on selected
- Tap to select and close

### 3. Speeding Threshold Options:

- At speed limit
- 5 over
- 10 over
- 15 over

**Purpose:** When to trigger speeding alert

**Default:** At speed limit

### 4. Toggle Switches

**Show on map:**

- Switch component
- Toggles speedometer visibility on map screen

**Alert when speeding:**

- Switch component
- Enable/disable audio/visual alerts when over threshold

**Code:**

```
const [showOnMap, setShowOnMap] = useState(true);
const [alertWhenSpeeding, setAlertWhenSpeeding] = useState(true);
```

```
<Switch
 value={showOnMap}
 onChange={setShowOnMap}
 trackColor={{ false: colors.border, true: colors.primary + "80" }}
 thumbColor={showOnMap ? colors.primary : colors.muted}
```

/>

**Section Header:**

- "Speed limit" text in gray
- Separates section visually

**Modal Pattern:**

- Consistent with other modals in app
- Slide-up animation
- Back and close buttons
- List of options with checkmarks

## Navigation Hierarchy

// App.js Navigation Structure

QueryClientProvider

└─ NavigationContainer

└─ RootStack (Stack Navigator)

└─ Auth Screens

└─ Splash

└─ Onboarding

└─ SignIn

└─ SignUp

└─ ForgotPassword

└─ ForgotPasswordCode

└─ ResetPassword

└─ PasswordChanged

└─

└─ MainTabs (Tab Navigator)

└─ Map Tab

└─ Alerts Tab

└─ Search Tab

└─ Settings Tab

└─ Stack Navigator

└─ SettingsMenu

└─ GasStation

└─ PreferredBrand

└─ Speedometer

## Navigation Functions

- Navigate to screen: `navigation.navigate("ScreenName");`
- Navigate with parameters: `navigation.navigate("GasStation", { selectedBrand: "Shell" });`
- Go back: `navigation.goBack();`
- Replace (no back): `navigation.replace("SignIn");`
- Access route params: `const { selectedBrand } = route.params;`

## State Management Patterns

### Local State (useState)

#### Used for:

- Form inputs
- Modal visibility
- Selection states
- Toggle switches

```
const [email, setEmail] = useState("");
const [showModal, setShowModal] = useState(false);
const [selectedOption, setSelectedOption] = useState("default");
```

### Navigation State (Route Params)

#### Used for:

Passing data between screens

Maintaining selections across navigation

// Sending

```
navigation.navigate("Screen", { data: value });
```

// Receiving

```
const { data } = route.params;
```

```
// Listening for changes
```

```
useEffect(() => {
 if (route.params?.data) {
 setState(route.params.data);
 }
}, [route.params?.data]);
```

### **Global State (Future: Context API or Redux)**

Planned for:

- User authentication state
- User preferences
- App settings
- Weather data cache

### **Code Examples and Common Patterns**

#### **Pattern 1: Screen with Modal**

```
export default function ExampleScreen({ navigation }) {
 const [showModal, setShowModal] = useState(false);
 const [selectedOption, setSelectedOption] = useState("Default");
 const options = [
 { id: "1", label: "Option 1" },
 { id: "2", label: "Option 2" },
];
```

```
 const handleSelect = (option) => {
 setSelectedOption(option);
 setShowModal(false);
```

```

};
return (
 <View style={styles.container}>
 <Pressable onPress={() => setShowModal(true)}>
 <Text>{selectedOption}</Text>
 </Pressable>

 <Modal
 visible={showModal}
 transparent={true}
 animationType="slide"
 onRequestClose={() => setShowModal(false)}
 >
 <View style={styles.modalOverlay}>
 <View style={styles.modalContent}>
 {options.map((option) => (
 <Pressable
 key={option.id}
 onPress={() => handleSelect(option.label)}
 >
 <Text>{option.label}</Text>
 {selectedOption === option.label && (
 <Icons name="checkmark" size={24} color={colors.primary} />
)}
 </Pressable>
))}
 </View>
 </View>
 </Modal>
 </View>
);
}

```

### Pattern 2: Form Validation

```
const [email, setEmail] = useState("");
const [emailError, setEmailError] = useState("");
const [emailValid, setEmailValid] = useState(false);

const validateEmail = (text) => {
 const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 const isValid = re.test(text);
 setEmailValid(isValid);
 setEmailError(isValid ? "" : "Invalid email format");
 return isValid;
};

<WDInput
 label="Email"
 value={email}
 onChangeText={(text) => {
 setEmail(text);
 validateEmail(text);
 }}
 showValidation={true}
 isValid={emailValid}
/>
{emailError ? <Text style={styles.error}>{emailError}</Text> : null}
```

### Pattern 3: Toggle Switch

```
const [isEnabled, setIsEnabled] = useState(false);

<View style={styles.settingItem}>
 <Text>Enable Feature</Text>
 <Switch
```

```

 value={isEnabled}
 onChange={setIsEnabled}
 trackColor={{ false: colors.border, true: colors.primary + "80" }}
 thumbColor={isEnabled ? colors.primary : colors.muted}
 />
</View>

```

#### Pattern 4: Navigation with Data

// Screen A - Sending

```

navigation.navigate("ScreenB", {
 userId: "123",
 userName: "John Doe"
});

```

// Screen B - Receiving

```

export default function ScreenB({ route, navigation }) {
 const { userId, userName } = route.params;

 return (
 <View>
 <Text>User: {userName}</Text>
 <Text>ID: {userId}</Text>
 </View>
);
}

```

#### Pattern 5: useEffect for Side Effects

// Run once on mount

```

useEffect(() => {
 fetchData();

```

```

}, []);

// Run when dependency changes
useEffect(() => {
 if (route.params?.selectedBrand) {
 setSelectedBrand(route.params.selectedBrand);
 }
}, [route.params?.selectedBrand]);

// Cleanup function
useEffect(() => {
 const subscription = subscribeToUpdate();
 return () => {
 subscription.unsubscribe();
 };
}, []);

```

## Styling Best Practices

### Using StyleSheet.create

```

const styles = StyleSheet.create({
 container: {
 flex: 1,
 backgroundColor: colors.bg,

```

```
},
header: {
 padding: spacing.lg,
 backgroundColor: colors.surface,
},
title: {
 ...type.h2,
 color: colors.text,
},
});
```

### **Dynamic Styling**

```
<View style={[
 styles.button,
 isActive && styles.buttonActive,
 isDisabled && styles.buttonDisabled
]}>
```

### **Conditional Styling**

```
style={[
 styles.input,
 isFocused && { borderColor: colors.primary, borderWidth: 2 },
 hasError && { borderColor: colors.danger }
]}
```

### **Future Integration Points**

#### **AWS Backend Connection**

#### **Authentication:**

```
// Replace mock navigation with AWS Cognito
import { Auth } from 'aws-amplify';

const handleSignIn = async () => {
 try {
 const user = await Auth.signIn(email, password);
 navigation.replace("MainTabs");
 } catch (error) {
 setError(error.message);
 }
};
```

### **Data Fetching:**

```
import { useQuery } from '@tanstack/react-query';

const { data: alerts } = useQuery({
 queryKey: ['weatherAlerts'],
 queryFn: () => fetch('/api/alerts').then(res => res.json())
});
```

### **Real-time Updates**

```
// WebSocket connection for live alerts
useEffect(() => {
 const ws = new WebSocket('wss://api.weatherdriver.com/alerts');

 ws.onmessage = (event) => {
 const newAlert = JSON.parse(event.data);
 setAlerts(prev => [newAlert, ...prev]);
 };
});
```

```
 return () => ws.close();
 }, []);
```

## 11. Testing Structure

### Unit Tests

**Location:** Implementation/client/\_\_\_tests\_\_\_/

#### Token Tests:

```
// tokens.test.js
import { colors, spacing, type } from '../src/theme/tokens';

describe('Design Tokens', () => {
 test('colors are defined', () => {
 expect(colors.primary).toBe("#20B2AA");
 });

 test('spacing values are numbers', () => {
 expect(typeof spacing.lg).toBe('number');
 });
});
```

## References

scrumstudy.com. (2025, September 15). *Scrum Phases and Processes*. Retrieved from [www.scrumstudy.com: https://www.scrumstudy.com/whyscrum/scrums-phases-and-processes](https://www.scrumstudy.com/whyscrum/scrums-phases-and-processes)

U.S. Department of Transportation, Federal Highway Administration. (2025, July 15). *How Do Weather Events Affect Roads*. Retrieved from [ops.fhwa.dot.gov: https://ops.fhwa.dot.gov/weather/roadimpact.htm#:~:text=On%20average%2C%20over%203%2C800%20people,during%20rain%20or%20mist%20conditions](https://ops.fhwa.dot.gov/weather/roadimpact.htm#:~:text=On%20average%2C%20over%203%2C800%20people,during%20rain%20or%20mist%20conditions).